

Linguaggi di Programmazione 1

Prof. Marco Faella
Anno 2024-2025

MARTA PAGLIUSO

1 Lezione-01

Linguaggio di programmazione: è un linguaggio che è usato per esprimere (mediante un programma) un algoritmo con il quale un processore può risolvere un problema.

Programma: è la codifica di un algoritmo in un linguaggio di programmazione.

Processore: è la macchina (reale o virtuale) che eseguirà l'algoritmo descritto dal programma. Non si deve intendere come un singolo oggetto, ma come una *architettura di elaboratori*.

È uso comune intendere come linguaggi di programmazione solo quelli **computazionalmente completi**, cioè quelli che possono esprimere *qualunque funzione calcolabile*. Vengono detti **general purpose** e, tecnicamente, devono poter simulare qualunque macchina di Turing.

Sono completi solo quelli che riescono ad esprimere anche programmi di cui non è decidibile la terminazione.

- SQL non è un linguaggio completo, perché la terminazione dei programmi è sempre decidibile.
- HTML non è un linguaggio completo, per lo stesso motivo di SQL.

1.1 Macchine Astratte

Macchina Astratta: Dato un linguaggio di programmazione L , una macchina astratta per L (in simboli M_L) è un qualsiasi insieme di strutture dati e algoritmi che permettano di memorizzare ed eseguire programmi scritti in L .

Come illustrato nella prossima figura, una generica macchina astratta M_L è composta da una *memoria* e da un *interprete*. La memoria serve per immagazzinare dati e programmi mentre l'interprete è il componente che esegue le istruzioni contenute nei programmi.

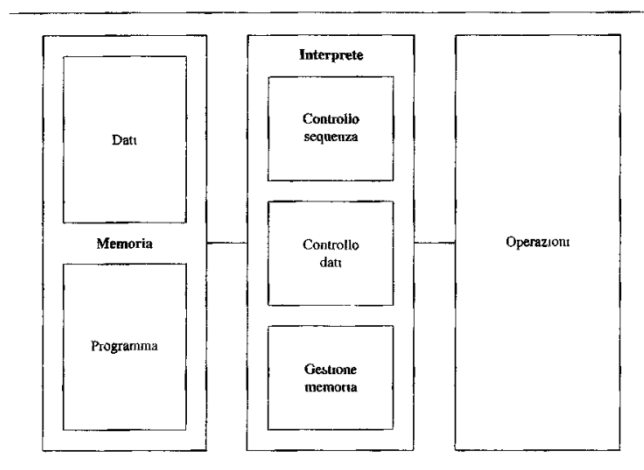


Figura 1.1 La struttura di una macchina astratta.

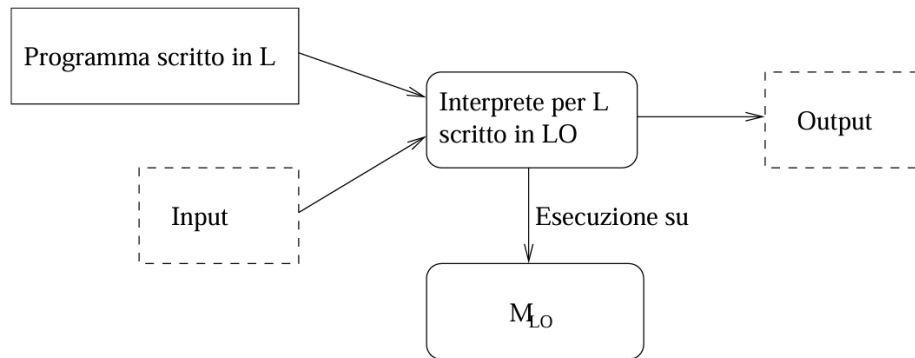
Le tecnologie di realizzazione di una macchina astratta sono:

- **Realizzazione in Hardware:** normalmente, quando si scrive un programma in L , un altro programma (chiamato "interprete" o "compilatore") lo traduce in un linguaggio che il computer capisce, che è il suo "linguaggio macchina". Quello che si fa è realizzare, mediante dispositivi fisici quali memorie, reti aritmetico-logiche, bus, ecc. una macchina fisica tale che il suo linguaggio macchina coincida con L .
- **Simulazione mediante Software:** consiste nella realizzazione delle strutture dati e degli algoritmi di M_L mediante programmi scritti in un altro linguaggio L' , che possiamo supporre già implementato (ad esempio Java Virtual Machine).
- **Simulazione (emulazione) mediante Firmware:** Concettualmente questa soluzione è simile alla simulazione mediante software: in entrambi i casi M_L è simulata mediante opportuni programmi che sono poi eseguiti da una macchina fisica. Tuttavia, nel caso della emulazione firmware tali programmi sono microprogrammi invece che programmi di un linguaggio di alto livello. È una soluzione più flessibile e semplice - economicità di progetto;

Per passare da un linguaggio a una macchina astratta esistono i seguenti traduttori:

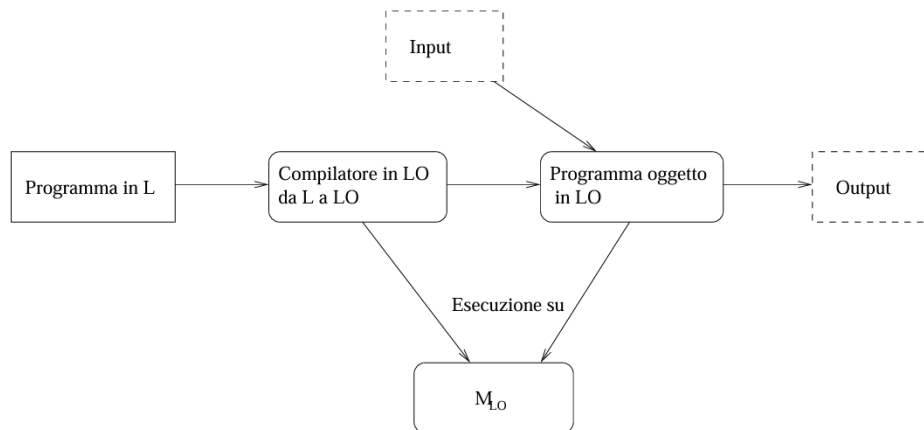
- **Interpreti:** traducono il linguaggio un costrutto alla volta. Hanno come pro un'iterazione più snella.
- **Compilatori:** prima traducono l'intero programma, poi la traduzione può essere eseguita (anche più volte). Hanno come pro la velocità di esecuzione finale e più controlli (e in anticipo).

Interpretazione pura

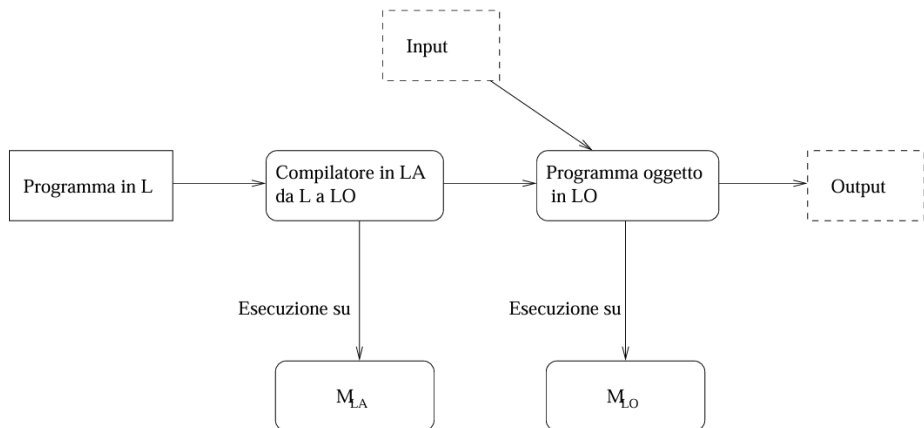


E' il caso dei linguaggi di scripting: bash, awk, Python, etc.

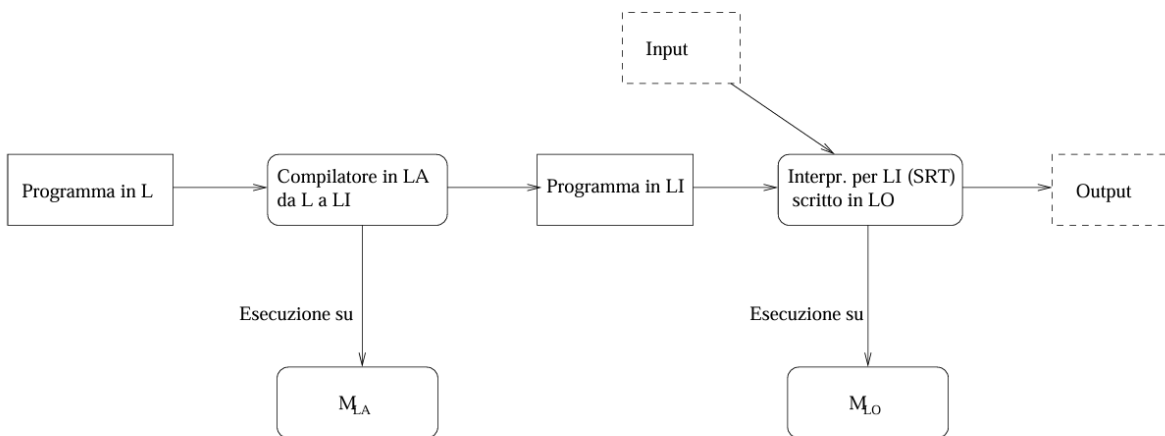
Compilazione pura (caso semplice)



Compilazione pura (caso più generale)



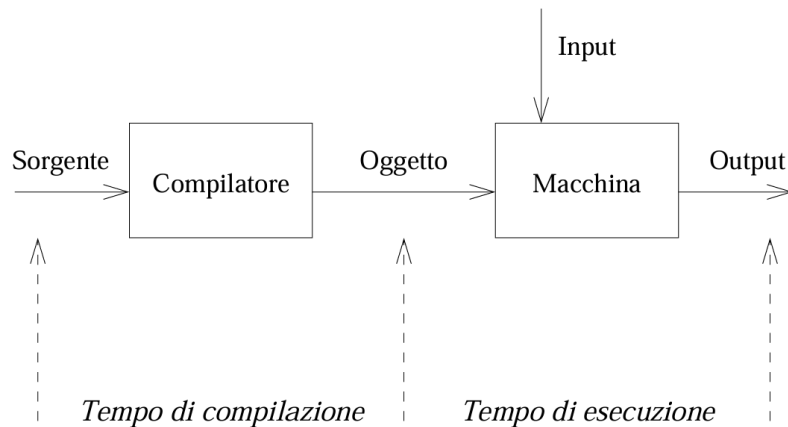
Compilazione per macchina intermedia



- L = Java (linguaggio sorgente)

- *LA* = Java (linguaggio del compilatore)
- *LI* = Bytecode Java (linguaggio intermedio)
- *LO* = C/C++ (linguaggio interprete JVM)

Compilazione ed esecuzione



Le componenti di un linguaggio di programmazione sono:

- **Grammatica:** anche detta sintassi;
- **Semantica:** *qual è il significato di ciascuna istruzione/costrutto?*
- **Pragmatica:** *come si usa efficacemente il linguaggio?*
- **Implementazione:** *come si realizza una macchina astratta per questo linguaggio?*

Le proprietà dei linguaggi sono:

- **Semplicità:**
 - Sintattica: unica rappresentabilità di ogni concetto.
 - Semantica: minimo numero di concetti e strutture.
- **Astrazione:**
 - Dati: nascondere i dettagli (incapsulamento, information hiding)
 - Funzioni e procedure: dividere il programma.
 - Moduli, pacchetti, etc...: favorire la modularità.
- **Espressività**
- **Ortogonalità:** poche eccezioni alle regole del programma.
- **Portabilità**

1.2 Paradigmi computazionali

Paradigma imperativo: paradigma secondo cui un programma viene inteso come un insieme di istruzioni, ciascuna delle quali può essere pensata come un "ordine" che viene impartito alla macchina. I valori vengono calcolati cambiando lo stato del programma attraverso delle assegnazioni.

Paradigma funzionale: pone maggior accento sulla definizione di funzioni. Un programma funzionale è immutabile: i valori non vengono trovati cambiando lo stato del programma, ma costruendo nuovi stati a partire dai precedenti.

Paradigma logico: programma come descrizione logica di un problema. Esecuzione analoga a processi di dimostrazione di teoremi.

Paradigma Orientato ad oggetti: programma costituito da oggetti che scambiano messaggi.

Paradigma parallelo: programmi che descrivono entità distribuite che sono eseguite contemporaneamente ed in modo asincrono.

Il paradigma di appartenenza può influenzare radicalmente il modo in cui si risolve il problema. In linguaggi dello stesso paradigma, lo stesso problema ha soluzioni strutturalmente identiche: imparato a risolvere un problema in un linguaggio, lo si sa risolvere in tutti i linguaggi dello stesso paradigma.

Il paradigma non è l'unico aspetto determinante. Altri esempi sono:

- Il sistema di tipi supportato;
- Eventuale supporto alle eccezioni;
- Presenza di garbage collection;
- Modello di concorrenza e sincronizzazione;
- ...

2 Lezione-02

2.1 Modello Imperativo

La **memoria** consiste in un insieme di "contenitori di dati" associati ai valori in essi contenuti. Dunque (concettualmente):

Memoria: è una funzione da uno spazio di locazioni ad uno spazio di valori:

$$\text{mem} : \text{Indirizzi} \longrightarrow \text{Valori}$$

$\text{mem}(\text{loc}) = \text{"valore contenuto nella locazione loc"}$.

Invece di $\text{exp} \longrightarrow \text{exp} + \text{exp}$ scriveremo $\langle \text{exp} \rangle ::= \langle \text{exp} \rangle + \langle \text{exp} \rangle$.

La sintassi dell'assegnazione è:

```
1 <assegnazione> ::= <name> <assignment-operator> <expression>
```

- $\langle \text{name} \rangle$ rappresenta la locazione dove viene posto il risultato
- In $\langle \text{expression} \rangle$ sono specificati una computazione e i riferimenti ai valori necessari alla computazione. Il valore di $\langle \text{expression} \rangle$ va memorizzato all'indirizzo rappresentato da $\langle \text{name} \rangle$.

Un esempio in Pascal:

```
1 a := b+c;
```

Il modello imperativo è il più vicino agli elaboratori "standard". Ogni unità di esecuzione consiste di 4 passi:

- Ottenere indirizzi delle locazioni di operandi e risultato;
- Ottenere dati di operandi da locazioni di operandi;
- Valutare il risultato;
- Memorizzare il risultato in locazione risultato.

In sostanza, un assegnamento. Il modello imperativo usa dei nomi come astrazione di indirizzi di locazioni di memoria (variabili).

L'**ambiente** comprende un insieme di nomi (di variabili, parametri, funzioni,...) associati a qualcosa da cui si può risalire al valore della variabile o del parametro. Concettualmente l'ambiente è una funzione da un insieme di identificatori a un insieme di...? (il punto interrogativo perché il codominio dipende dal paradigma computazionale del linguaggio).

Spieghiamo meglio questo concetto:

- I nostri programmi usano nomi per riferirsi alle cose. Ad esempio, se scriviamo $x = 5$, " x " è un nome. Se definiamo una funzione $\text{somma}(a, b)$, " somma ", " a " e " b " sono nomi. L'ambiente tiene traccia di tutti questi nomi.

- Ogni nome è collegato a qualcosa. Nel caso di $x = 5$, il nome "x" è associato al valore 5. Se è una funzione, il nome della funzione è associato al codice che la funzione deve eseguire. In pratica, l'ambiente permette di sapere cosa "significa" ogni nome in un dato momento.
- Abbiamo detto che concettualmente l'ambiente è una funzione da un insieme di indentificatori (nomi) a un codominio indefinito. Quest'ultimo è tale perché non sempre, preso un nome, viene restituito lo stesso valore. Ad esempio:
 - In linguaggi semplici (imperativi), il codominio potrebbe essere semplicemente un valore (il contenuto di una variabile).
 - In linguaggi più complessi (funzionali o orientati agli oggetti), il codominio potrebbe essere un indirizzo di memoria, un blocco di codice, un altro ambiente (per scope annidati), o anche un oggetto intero.

Un esempio di ambiente nel paradigma imperativo è la funzione **env** che associa gli identificatori a locazioni di memoria, le quali, a loro volta, sono associate (funzione **mem**) al contenuto di memoria.

$$\text{env} : \text{Nomi} \longrightarrow \text{Indirizzi}$$

$\text{env}(x) = \text{"indirizzo della variabile di nome } x\text{"}$. Il valore di una variabile è $\text{mem}(\text{env}(x))$.

Nel paradigma funzionale (puro) il valore associato a un nome non cambia nel tempo. Quindi la funzione **env** associa direttamente gli identificatori al contenuto della memoria. In questo caso:

$$\text{env} : \text{Nomi} \longrightarrow \text{Valori}$$

Il valore di una variabile x è $\text{env}(x)$. Nel paradigma funzionale non esiste la funzione **mem**.

L'assegnazione è un'operazione che modifica l'ambiente. Un esempio:

```
1  x := x + 1;
```

il valore dell'espressione a destra va salvato nella variabile a sinistra. Quindi:

- La x di sinistra indica la locazione associata al nome (cioè $\text{env}(x)$).
- La x di destra indica il valore della variabile (cioè $\text{mem}(\text{env}(x))$).

Possiamo quindi tradurre l'assegnazione così:

```
1  In env(x) store mem(env(x)) + 1
```

2.2 Data Object e legami

Ogni variabile, parametro, oggetto, ... può essere rappresentato da un **data object**.

Data Object: è una quadrupla (L, N, V, T) dove:

- L : locazione;
- N : nome;
- V : valore;
- T : tipo.

Il **legame** (o *binding*) è la determinazione di una delle componenti. Nel paradigma imperativo, dato un data object (L, N, V, T) , per il legame di locazione vale $L = \text{env}(N)$, mentre per il legame di valore vale $V = \text{mem}(\text{env}(N))$.

Un **legame** può essere stabilito o modificato in diverse fasi:

- Durante la compilazione (*compile time*)
- Durante il caricamento in memoria (*load time*)
- Durante l'esecuzione (*run time*)

Il location binding avviene durante il caricamento in memoria oppure a run time. Il name binding avviene durante la compilazione, nell'istante in cui il compilatore incontra una dichiarazione. Il type binding avviene (di solito) durante la compilazione, nell'istante in cui il compilatore incontra una dichiarazione di tipo.

2.3 Legami di tipo

Per definizione è correlato al legame di valore (ad es. il tipo di una variabile e il tipo del suo valore devono corrispondere). Ogni volta che il legame di valore viene modificato, occorrerebbe controllare (*type checking*) la coerenza del legame di tipo.

Dinamicamente tipizzato: un linguaggio è dinamicamente tipizzato se il legame (e le variazioni del legame) e di conseguenza anche il controllo di coerenza (se avviene) avvengono durante l'esecuzione (es. Python).

Staticamente tipizzato: un linguaggio è staticamente tipizzato se il legame avviene durante la compilazione. In questo caso il controllo di coerenza (se avviene) può avvenire in entrambe le fasi.

2.3.1 Type Checking

È il meccanismo di controllo di coerenza di coppia dei legami valore-tipo. Può avvenire:

- a) Durante la compilazione;
- b) Durante l'esecuzione;
- c) Per nulla.

Fortemente tipizzato: un linguaggio è fortemente tipizzato se il controllo di coerenza avviene sempre (es. Java).

Debolmente tipizzato: un linguaggio è debolmente tipizzato se il controllo di coerenza può non avvenire affatto in numerosi casi.

C, ad esempio, è debolmente tipizzato:

1. Supporta operazioni di *casting*;
2. Supporta conversioni implicite (senza `cast`).

2.3.2 Linguaggio perfetto

Linguaggio perfetto: è un linguaggio Turing completo in cui il type checking avviene completamente durante la compilazione e in cui il compilatore non genera mai più errori del necessario.

Se esistesse sarebbe staticamente e fortemente tipizzato, ma non può esistere. Se esistesse, allora il compilatore, per essere capace di decidere la correttezza dell'ultima assegnazione in:

```
1  int x;  
2  P;  
3  x = "pippo";
```

dove P è un generico programma, dovrebbe essere capace di decidere la terminazione di P, quindi il linguaggio non sarebbe Turing completo, contro l'ipotesi.

2.4 Blocchi di istruzioni

Scope: si definisce *scope* o *ambito di validità* di un legame, l'insieme degli enunciati del programma in cui quel legame è valido. I linguaggi moderni limitano lo scope usando blocchi di codice (che possono prendere la forma di procedure, funzioni, classi, etc...)

A scopo didattico, introduciamo un semplice pseudo-linguaggio di blocchi di codice. Un blocco contiene due parti:

- Una sezione dedicata ai nomi (cioè variabili);
- una sezione che comprende gli enunciati sui quali hanno validità quei legami di nome.

Esempio:

```
1  ...  
2  BLOCK A;  
3    DECLARE I;  
4  BEGIN A  
5    ...          {I DEL BLOCCO A}  
6  END A;  
7  ...
```

Sono essenziali due tipi di ambito di validità (scoping):

- **Statico:** Blocchi annidati vedono e usano i legami dei blocchi più esterni (legami non locali) e possono aggiungere legami locali o sovrapporne di nuovi. Immaginiamo che la "visibilità" delle cose sia decisa solo da come sono scritte le scatole una dentro l'altra nel nostro codice. Se abbiamo una scatola piccola dentro una scatola grande, tutto ciò che è nella scatola grande è visibile e utilizzabile anche dalla scatola piccola. Questi sono i "legami non locali" (cioè, non definiti nella scatola piccola ma in una esterna).

La scatola piccola può anche definire le sue proprie cose (legami locali) che valgono solo lì dentro. Oppure, se definisce qualcosa con lo stesso nome di una cosa che vede fuori, la sua definizione locale ha la precedenza (sovrascrive quella esterna per quel blocco specifico). La maggior parte dei linguaggi moderni come Python, JavaScript, Java, C++, C# usano lo scoping statico. È più facile da capire e prevedere come si comporterà il codice.

- **Dinamico:** Concetto qui esaminato solo in relazione ai blocchi annidati, ma che assume il proprio senso maggiore quando vi sono procedure chiamanti e chiamate. In questo caso la procedura chiamata vede e usa i legami visti e usati dalla procedura chiamante.

Qui la "visibilità" delle cose non è decisa da come sono scritte le scatole una dentro l'altra, ma da come le funzioni si chiamano l'un l'altra durante l'esecuzione del programma.

Il vero significato dello scoping dinamico si capisce meglio quando una funzione (A) ne chiama un'altra (B). Se la funzione A chiama la funzione B, la funzione B può "vedere" e usare le variabili e le definizioni che erano visibili e in uso dalla funzione A al momento della chiamata. Non importa dove B è stata definita nel codice, ma da dove è stata chiamata.

Lo scoping dinamico è molto meno comune nei linguaggi moderni (esisteva in alcuni linguaggi più vecchi come Lisp in alcune implementazioni, o in linguaggi di scripting come Perl 4).

Mascheramento: Si parla di mascheramento (o *shadowing*) quando un blocco interno ridefinisce un nome che era già definito in un blocco esterno. In tal caso, la nuova definizione nasconde (maschera) quella vecchia, finché non esce dal blocco interno.

```
1 void f(int n) {
2     int i = 0;
3     {
4         int n = 0; // maschera n
5         n++;      // incrementa la nuova n
6     }
7     n--;          // decrementa la vecchia n
8 }
```

```
1 PROGRAM P;
2     DECLARE X,Y;
3 BEGIN P
4     ...           {X e Y da P}
5     BLOCK A;
6         DECLARE X,Z;
7     BEGIN A
8         ...       {X e Z da A, Y da P}
9     END A;
10    ...           {X e Y da P}
11 END P;
```

2.5 Legami di locazione

Stabilire un legame di locazione si chiama **allocare memoria**. Se una variabile è in scope, deve avere un legame di locazione, ma non vale il viceversa: un legame di locazione può essere attivo mentre la variabile non è il scope (es. per mascheramento).

L'allocazione può essere:

- **Statica:** l'indirizzo viene fissato a load-time e tale allocazione vale per tutta l'esecuzione;

- **Dinamica:**

- Su stack: l'indirizzo viene fissato a runtime su richiesta e viene rilasciato al termine del blocco corrente;
- Su heap: l'indirizzo viene fissato a runtime su richiesta e viene rilasciato su richiesta oppure automaticamente quando non più utilizzato (garbage collection).

Si dice *allocazione statica di memoria* quando il legame di locazione è fissato e costante a loadtime. Tipico delle variabili globale (se previste dal linguaggio).

```
1  int g;      // alloc. statica e scope globale
2
3  void f(int n) {
4      static int i;    // alloc. statica e scope locale
5      int i;
6      ...
7  }
```

Si dice *allocazione dinamica tramite stack* quando il legame di locazione è creato all'inizio dell'esecuzione di un blocco e viene rilasciato automaticamente alla fine del blocco. Essa è realizzata attraverso il **Record di Attivazione** (RdA) di un blocco.

Il contenuto dell'RdA nel caso di un blocco anonimo o *in-line*:

- Puntatore di catena dinamica (link all'RdA precedente);
- Ambiente locale (variabili locali e spazio per risultati intermedi)

Esempio di allocazione dinamica su stack in C:

```
1  int g;
2
3  void f (int n) {      // n: allocazione dinamica su stack e scope locale
4      static int a;
5      int i;            // i: allocazione dinamica su stack e scope locale
6      {
7          int j;        // j: allocazione dinamica su stack e scope locale
8      }
9      ...
10 }
```

In ogni momento dell'esecuzione lo **stack** (o pila) **di esecuzione** contiene gli RdA "attivi":

1. Il top dello stack contiene l'RdA del blocco correntemente in esecuzione;
2. Ogni volta che si entra in un blocco, l'RdA del blocco viene creato e posto sullo stack (push);
3. Ogni volta che si esce da un blocco, viene eliminato l'RdA in cima allo stack (pop).

[Vedi esempio nelle slide]

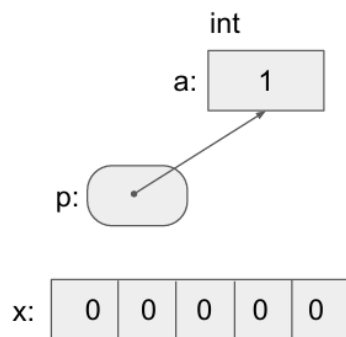
2.6 Richiami C/C++ - Diagramma di Memory Layout - Notazione Data Object

Vediamo un esempio di data object in C (tra parentesi c'è il data object corrispondente alla riga di codice):

```
1  int a = 1;          (loc1, a, 1, int)
2  int *p = &a;        (loc2, p, loc1, int*)
3  int x[5] = {0, };   (loc3, x, {0,0,0,0,0}, int[])
```

Il **diagramma di Memory Layout** mostra le allocazioni di memoria contigue. Il valore di puntatori e riferimenti è rappresentato da frecce. Non si distingue tra stack e heap.

Il diagramma dell'esempio precedente sarebbe:



2.6.1 Richiami di C

Dichiarazione di un nuovo tipo di dati

```
1  struct Person {
2      char name[100];
3      int age;
4  };
```

Definizione di variabile di tipo **Person**, non inizializzata. Se globale, viene automaticamente inizializzata a zero (tutti zeri, anche i **char**).

```
1  struct Person p;
```

Dichiarazione del tipo e **definizione** della variabile, congiunte:

```
1  struct Person {
2      char name[100];
3      int age;
4  } p;
```

Definizione e inizializzazione (vecchio stile):

```
1  struct Person p1 = {"Pippo", 30};
```

Nel nuovo stile C99 invece:

```
1 struct Person p2 = {
2     .name = "Pippo",
3     .age = 30
4 };
```

Codice C:

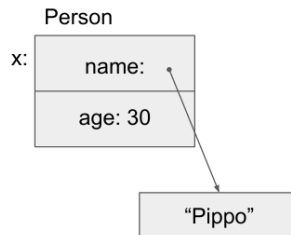
```
struct Person {
    char *name;
    int age;
} x = {
    .name = "Pippo",
    .age = 30
};
```

`sizeof(Person) → 16`

Perché 16?

alignment e padding

Memory layout:



Data Objects:

principali:

(loc1, x, {loc2, 30}, struct Person)

(loc2, anonimo, "Pippo", const char[])

aggiuntivi:

(loc1, x.name, loc2, char*)

(loc1+8, x.age, 30, int)

ipotizzando che `sizeof(char*)` sia 8

2.6.2 Java

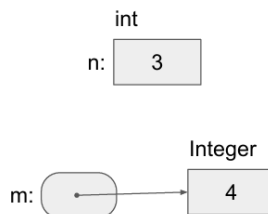
Codice Java:

```
int n = 3;
```

```
Integer m = 4;
```

Nota: grazie all'*autoboxing*, la seconda istruzione è equivalente a
`Integer m = Integer.valueOf(4)`

Memory layout:



Data Objects:

(loc1, n, 3, int)

(loc2, anonimo, 4, Integer)

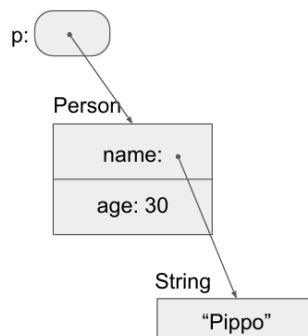
(loc3, m, loc2, "riferimento a Integer")

Codice Java:

```
class Person {
    String name;
    int age;
    ...
}
```

```
Person p = new
Person("Pippo", 30);
```

Memory layout:



Data Objects:

(loc1, anonimo, ..., Person)

(loc2, p, loc1, "riferimento a Person")

(loc3, anonimo, "Pippo", String)

Vediamo i **tipi riferimento** in Java:

```
1 Person p = new Person ("Pippo", 30);
```

Il primo `Person` indica il tipo "riferimento a `Person`", mentre il secondo indica il tipo `Person`.

Quando il nome di un tipo `T` (non primitivo) viene usato per creare un oggetto (cioè, `new T(...)`), crea un oggetto della classe `T`. Quando il nome di un tipo `T` (non primitivo) viene usato per dichiarare una nuova variabile, indica il tipo "riferimento a `T`".

2.6.3 C++

Le **classi** sono dichiarate in questo modo:

```
1 class Person {
2     private:
3         string name;
4         int age;
5     public:
6         Person (string n, int a): name(n), age(a){}
7
8     };
```

L'**allocazione automatica** di un oggetto `Person` (su stack se locale, su segmento statico se globale):

```
1 Person p = Person("Pippo", 30);
2 // oppure
3 Person p("Pippo", 30);
4 // oppure
5 Person p {"Pippo", 30};
```

Un **riferimento** è un alias come *puntatore costante*. Sintatticamente non si comporta come un puntatore, ma come l'oggetto puntato. L'uso tipico: passaggio dei parametri per riferimento:

- Se il metodo non deve modificare l'argomento, si usa `const T&`;
- Se il metodo modifica l'argomento, si usa `T&`.

Con puntatore:

```
1 void f(struct Big *big) {
2     cout << big->field;
3     cout << big;        // indirizzo della struct
4     cout << *big;        // contenuto della struct
5     big->field = 0;
6     big = nullptr;
7 }
```

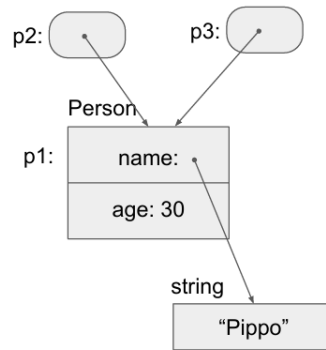
Con riferimento:

```
1 void f(struct Big &big) {
2     cout << big.field;
3     cout << big;        // contenuto della struct
4     cout << &big;        // indirizzo della struct
5     big.field = 0;
6     big = nullptr;      // errore di compilazione
7 }
```

Codice C++:

```
class Person {  
    string name;  
    int age;  
    ...  
};  
  
Person p1 =  
Person("Pippo", 30);  
  
Person &p2 = p1;  
Person *p3 = &p1;
```

Memory layout:



Data Objects:

(loc1, p1, ..., Person)
(loc2, p2, loc1, riferimento a Person)
(loc3, p3, loc1, puntatore a Person)
(loc4, anonimo, "Pippo", string)

3 Lezione 03

3.1 Procedure come astrazioni

Procedure: sono astrazioni di parti di programma in unità di esecuzione più piccole (invocazioni), in modo da nascondere i dettagli irrilevanti ai fini del loro (ri-)uso.

I vantaggi sono:

- Programmi più semplici da leggere, scrivere e modificare; progettazione top-down.
- Unità di programmi indipendenti o con dipendenze ben specificate a livello più alto.
- Riusabilità e riduzione di errori.

Si definisce **astrazione procedurale** la rappresentazione di un'unità di esecuzione attraverso un'altra unità più semplice (invocazione). In pratica è la rappresentazione di un blocco attraverso un enunciato o un'espressione.

La dichiarazione di una procedura causa la generazione di un legame tra il nome della procedura e il corpo della procedura. Chiamiamo questo legame un oggetto "**Tipo procedura**". Questo legame viene stabilito durante la compilazione.

L'invocazione di una procedura causa la generazione di un legame tra un Tipo procedura e un Record di Attivazione. Chiamiamo questo legame un "**Oggetto procedura**". Il legame avviene durante l'esecuzione, nel momento in cui c'è l'invocazione della procedura. Ogni invocazione diversa della stessa procedura causa la generazione di un nuovo Oggetto procedura con lo stesso legame di tipo, ma con diverso RdA.

Analogamente a quanto avveniva con un blocco in-line, il **Record di Attivazione** rappresenta l'intero ambiente di esecuzione di una procedura o funzione. A differenza del blocco anonimo, la procedura o funzione:

- Ha un nome e può essere invocata in vari punti del programma;
- Può avere parametri;
- Può avere un valore di ritorno (se funzione).

L'RdA di una funzione contiene:

1. Puntatore di catena dinamica (link al precedente RdA);
2. Puntatore di catena statica (opzionale, serve per scoping statico);
3. Indirizzo di ritorno;
4. Indirizzo del risultato;
5. Parametri;
6. Ambiente locale.

3.2 Propagazione dei data object

L'**ambiente non locale** è l'insieme di tutti quei nomi a cui la funzione può riferirsi e che non sono dichiarati in quella funzione. La provenienza di questi data object propagati dipende dal linguaggio. Abbiamo tre tipi di propagazione (scoping).

Scoping statico: una variabile è risolta in base alla struttura in blocchi nel codice sorgente. Significa che il compilatore determina dove andare a cercare una variabile prima dell'esecuzione.

Scoping dinamico: una variabile è risolta in base alla sequenza di chiamate a runtime. Significa che il valore dipende da chi ha chiamato la funzione, non da dove è dichiarata nel codice.

È possibile anche non avere nessuna propagazione.

Lo scoping viene realizzato inserendo nell'RdA un puntatore all'RdA della funzione da cui vengono propagati i data object. Per lo scoping statico viene usato il puntatore di catena statico, mentre per lo scoping dinamico viene usato il puntatore di catena dinamica.

L'RdA di un blocco in-line non contiene il puntatore di catena statica (anche in presenza di scoping statico) perché viene eseguito sempre nel contesto del blocco in cui è contenuto sintatticamente. Non può "saltare" a un contesto lessicale diverso, a differenza di una funzione che può essere passata come parametro e chiamata da un altro punto del codice. Quindi l'eventuale puntatore di catena statica coinciderebbe con quello di catena dinamica.

Vediamo un esempio di scoping statico con procedure annidate:

```
1  program p;  
2    var a,b,c: integer;  
3  
4    procedure q;  
5      var a,c: integer;  
6      procedure r;  
7        var a: integer;  
8        begin r      // {variabili: a da r; b da p; c da q;  
9          ...         // procedure: q ed s da p; r da q}  
10       end r;  
11      begin q         // {variabili: a da q; b da p; c da q;  
12        ...           // procedure: q ed s da p; r da q}  
13      end q;  
14    procedure s;  
15      var b: integer;  
16      begin s         // {variabili: a da p; b da s; c da p;  
17        ...           // procedure: q ed s da p}  
18      end s;  
19    begin p           // {variabili: a, b, c da p;  
20      ...             // procedure: q ed s da p}  
21    end p.
```

Esempio di scoping dinamico:

```
1  program p;  
2    var a: integer;  
3    procedure q;  
4      begin q  
5        ...           // {vars: a da p o da r; procs: q, r da p}  
6      end q;  
7    procedure r;  
8      var a: integer;  
9      begin r  
10       ...           // {vars: a da r; procs: q, r da p}  
11     end r;  
12  begin p  
13    ...               // {vars: a da p; procs: q, r da p}  
14  end p;
```

3.3 Equivalenze di tipi

All'inizio esistevano solo i tipi elementari e nessuna gerarchia di classi. Poi sono stati introdotti i tipi user-defined. La validità di un assegnamento $x = y$ (o $x := y$) o un passaggio di parametri era stabilito dalla nozione di **type equivalence** adottata dal linguaggio.

Name equivalence: i tipi x e y devono avere lo stesso nome, cioè essere lo stesso tipo.

Structural equivalence: i tipi x e y devono avere la stessa rappresentazione interna. Apparentemente più snello e flessibile, in realtà aumenta la presenza di errori.

Pascal ha il name equivalence, mentre C/C++ entrambe (Quasi sempre structural tranne per le struct). Facciamo un esempio:

```
1  typedef int money;  
2  typedef int apples;  
3  
4  typedef struct {int val; } S1;  
5  typedef struct {int val; } S2;  
6  
7  int main () {  
8    money x = 0;  
9    apples y = 0;  
10   int z = x + y;    // non fa una piega  
11   S1 a;  
12   S2 b;  
13   a = b;           // errore di compilazione  
14 }
```

Questo perché verificare che due struct sono strutturalmente equivalenti richiede di verificare una proprietà chiamata **bisimulazione**.

Con l'avvento dei linguaggi orientati a oggetti, l'equivalenza viene rimpiazzata da **compatibilità/assegnabilità** (qualunque sottotipo di T è assegnabile a T). L'equivalenza è simmetrica, mentre l'assegnabilità (in Java) è anti-simmetrica.

Nei linguaggi O.O. più comuni la compatibilità è basata su nome piuttosto che struttura. Due classi con nomi diversi sono diverse, anche se hanno gli stessi attributi e gli stessi metodi.

Quindi l'evoluzione dei sistemi di tipi è stata:

1. Tipi di dato elementari;
2. Tipi user-defined (es. Pascal, C);
3. Interfacce e tipi di dato astratti (encapsulation, modificatori di accesso);
4. Gerarchie di tipi (linguaggi O.O.).

4 Lezione-04

4.1 Parametrizzazione di procedure

I **parametri** costituiscono il mezzo attraverso il quale le informazioni transitano esplicitamente tra l'unità chiamante e quella chiamata. Alcuni linguaggi dispongono di tre categorie:

- **Parametri in ingresso (IN)**: sono passati dall'unità chiamante alla unità chiamata al momento dell'invocazione;
- **Parametri di uscita (OUT)**: sono passati dall'unità chiamata alla unità chiamante al momento della terminazione della prima;
- **Parametri sia in ingresso sia in uscita (IN-OUT)**: servono a far transitare le informazioni in entrambe le direzioni.

Devono essere specificati in due punti:

- Nella definizione della procedura: *parametri formali*;
- Nelle invocazione della procedura: *parametri attuali* o *argomenti*.

Nei linguaggi staticamente tipati, nella definizione deve essere specificato il tipo dei parametri formali. Nell'invocazione è richiesta la compatibilità di tipo tra parametri formali e attuali (es. Java, C, C++, etc...).

Nei linguaggi dinamicamente tipati, i parametri formali non hanno alcun vincolo di tipo. Il legame di tipo si instaura a runtime allo stesso tipo dei parametri attuali (impossibile il type checking in compilazione) (es. Python). Il tipo dei parametri formali viene stabilito dal tipo dei parametri attuali a tempo di esecuzione.

Abbiamo due **metodi di associazione** tra parametri attuali e formali:

- **Per posizione**: a seconda della posizione relativa nella sequenza dei parametri;
- **Per nome**: il nome del parametro formale è aggiunto come prefisso al parametro attuale.

Un esempio di associazione per nome è (in Python):

```
1 def crea_persona(nome, eta, citta):
2     print(f"Nome: {nome}, Eta': {eta}, Citta': {citta}")
3
4     crea_persona(citta="Milano", nome="Bob", eta=25)
```

Un'ulteriore tecnica è la cosiddetta **associazione di default**. Essa permette di specificare valori di default per quei parametri formali che non sono stati legati a valori di parametri attuali. Un esempio in C++:

```
1 void print_error_msg(int line, string message = "Generic error") {}
```

I **parametri IN** possono essere realizzati in due modi:

1. **Per riferimento**: la locazione del parametro attuale diventa la locazione del formale. Si deve impedire (a tempo di compilazione, se possibile) la modifica all'interno della procedura;

2. **Per copia:** il valore del parametro attuale viene copiato in una nuova locazione, quella del parametro formale (il quale viene visto come variabile locale). La modifica è permessa perché valida solo nell'ambiente di esecuzione della procedura.

I **parametri OUT** possono essere realizzati:

1. Per riferimento;
2. Per copia (cioè per risultato). La copia avviene all'uscita.

Rappresentano dei risultati, quindi bisogna proibirne la lettura, ad esempio:

- Uso a destra di un assegnamento: se abbiamo una cosa del tipo `x = parametro_OUT`, significherebbe che si sta leggendo il suo valore prima che la funzione gli assegni un risultato.
- Se un parametro OUT venisse passato come parametro IN (Input) o IN-OUT (Input/Output) a un'altra funzione, ciò implicherebbe una lettura del suo valore.

Nota: nei nostri esercizi: errore di compilazione se un parametro OUT viene letto senza essere stato inizializzato.

Consideriamo il seguente programma:

```
1  program main (input, output);
2      var a,b,c: integer;
3
4      procedure p1 (OUT a,b: integer);
5      begin
6          a := a*b;
7          if (c/b) = a then a := 0 else a := 100
8      end;
9
10     begin
11         a := 5; b := 6; c := 7;
12         p1 (b, c);
13         writeln (a, b, c);
14     end.
```

Al momento dell'ingresso nella procedura, un parametro OUT è considerato non inizializzato. Il suo valore precedente (quello della variabile passata dal chiamante) viene ignorato o non è garantito che sia accessibile. Quindi il programma in esempio dà un errore di compilazione, per il rigo `a := a*b`; in cui si vuole leggere i valori di `a` e `b` prima che la procedura abbia assegnato loro un valore.

I **parametri IN-OUT** sono la combinazione dei due precedenti. Anch'essi possono essere realizzati:

1. Per riferimento: non ci sono limitazioni all'interno della procedura;
2. Per copia (per valore-risultato): avvengono due processi di copia, uno durante l'attivazione e uno al termine della procedura.

Per l'esempio precedente, se le variabili `a` e `b` fossero passate come IN-OUT, il programma non darebbe problemi di compilazione.

Aliasing: è la possibilità di riferirsi alla stessa locazione con nomi diversi. Nel passaggio dei parametri può causare notevoli problemi di interpretazione.

Per esempio:

```
1  program MAIN;
2      var
3          A: integer;
4      procedure TEST (X, Y: IN-OUT integer);
5      begin
6          X := A + Y;
7          writeln(A,X,Y)
8      end;
9  begin
10     A := 1;
11     TEST (A,A);
12     writeln(A)
13 end.
```

Se i parametri fossero realizzati per riferimento avremo come risultato:

```
1  2,2,2
2  2
```

Se invece fossero realizzati per copia:

```
1  1,2,1
2  1
```

Macro: è la generazione di un nuovo brano di codice sorgente in cui i nomi dei parametri sostituiscono i nomi dei parametri formali.

Ad esempio, data la seguente procedura:

```
1  procedure swap(a,b: integer);
2      var temp: integer;
3      begin
4          temp := a;
5          a := b;
6          b := temp;
7      end;
```

allora la chiama `swap(x,y)` viene sostituita dal seguente brano di codice:

```
1  temp := a;
2  a := b;
3  b := temp;
```

Le **funzioni** sono procedure che restituiscono un valore alla procedura chiamante. Sono realizzate:

- Utilizzato un'istruzione **return** per restituire esplicitamente il controllo al chiamante, inviando allo stesso tempo il valore di un'espressione;
- Oppure creando una *pseudo variabile* nell'ambiente locale della procedura chiamata. Tale variabile può essere solo modificata, non è possibile l'accesso in lettura.
- Oppure l'intero corpo della funzione è un'espressione (tipico dei linguaggi funzionali).

5 Java-01

Il linguaggio Java è staticamente tipizzato. "tipizzato" vuol dire che ad ogni espressione viene assegnato un tipo; "staticamente" vuol dire che il tipo di ogni espressione deve essere noto a tempo di compilazione.

I tipi base sono 8:

- boolean
- char
- byte(8 bit), short (16 bit), int(32 bit), long (64 bit)
- float (32 bit), double (64 bit)
- Inoltre, void è un tipo speciale usato solo come tipo di ritorno dai metodi.

Tra tipi base esistono le seguenti **conversioni implicite** (o **promozioni**):

- byte $\rightarrow$ short, short $\rightarrow$ int, int $\rightarrow$ long, long $\rightarrow$ float, float $\rightarrow$ double
- char $\rightarrow$ int

Queste conversioni sono transitive.

Alcune conversioni implicite possono portare a una *perdita di informazione*. Ad esempio:

```
1  int i = 1000000001;      // un miliardo e uno
2  float f = i;
```

Dopo queste istruzioni, `f` contiene un miliardo perché un float non ha abbastanza bit di mantissa per rappresentare le 10 cifre significative di `i`.

A differenza degli altri linguaggi orientati agli oggetti (ad es. C++), non esistono variabili che contengono oggetti, solo *riferimenti* ad oggetti, ovvero variabili che contengono l'indirizzo di un oggetto. I riferimenti in Java sono quindi simili ai puntatori del linguaggio C. Tuttavia, in Java sono molto più restrittivi:

- Niente aritmetica dei puntatori e conversioni tra puntatori e numeri interi;
- Niente doppi puntatori;
- Niente puntatori a funzioni.

Java prevede solo il **passaggio per valore**: sia i tipi base che i riferimenti sono passati per valore.

In virtù del polimorfismo, bisogna distinguere due tipi di ogni variabile ed espressioni di categoria "riferimento":

```
1  Animal a = new Dog("Lilli");
```

`Animal` è il tipo **dichiarato** (o statico) del riferimento `a`. Il tipo dichiarato è noto a tempo di compilazione e rimane invariato in tutto l'ambito di validità di quel riferimento.

`Dog` è il tipo **effettivo** (o dinamico) del riferimento `a`. In questo momento il tipo effettivo non è noto a tempo di compilazione, ma a quello di esecuzione. Il tipo effettivo può cambiare ogni volta che si assegna un nuovo valore a quel riferimento.

Proviamo adesso a simulare la fase di *type checking* del compilatore Java sul seguente frammento di codice:

```
1  int n;  
2  double d;  
3  d = (new Object()).hashCode()+n/2.0;
```

Cominciamo dalla parte destra dell'assegnazione e procediamo dalle sottoespressioni più piccole via via fino all'intera parte destra.

- La prima espressione che incontriamo è `new Object()`. Questa è per definizione di tipo "riferimento della classe `Object`", o per brevità, di tipo "`Object`".
- Passiamo poi all'espressione `(new Object()).hashCode()`. In questo contesto il punto denota la chiamata ad un metodo, quindi il tipo dell'espressione è il "tipo di ritorno del metodo", in questo caso `int`.
- L'espressione `n` ha chiaramente tipo `int`, mentre `2.0` è una costante di tipo `double`. Quindi i due operandi della visione `n/2.0` hanno diverso tipo. Il primo operando viene quindi promosso da `int` a `double` tramite conversione implicita. Complessivamente, la divisione ha a sua volta tipo `double`. Per lo stesso motivo, anche la somma avrà tipo `double`.
- A questo punto, il compilatore verifica che il tipo calcolato per il lato destro dell'assegnazione sia compatibile con il tipo del lato sinistro (`d`). In questo caso i due tipi coincidono perfettamente.

Per permettere il polimorfismo esiste una relazione binaria tra tipi, chiamata **relazione di sottotipo**.

Relazione di sottotipo: è definita dalle seguenti regole, in cui `T` ed `U` rappresentano tipi arbitrari (esclusi i tipi base):

1. `T` è sottotipo di se stesso;
2. `T` è sottotipo di `Object`;
3. Se `T` estende `U` oppure implementa `U`, `T` è sottotipo di `U`;
4. Il tipo nullo è sottotipo di `T`;
5. Se `T` è sottotipo di `U` allora `T[]` è sottotipo di `U[]`.

È facile verificare che questa relazione è riflessiva, simmetrica e transitiva. Pertanto essa è una *relazione d'ordine* sull'insieme dei tipi (non base).

La relazione di sottotipo permette di definire precisamente il comportamento dell'operatore `instanceof`.

instanceof: data un'espressione `exp` ed il nome di una classe o interfaccia `T`, l'espressione `exp instanceof T` restituisce vero se e solo se il tipo effettivo di `exp` non è nullo ed è sottotipo di `T`.

Nota: Il primo membro deve essere un'espressione di categoria "riferimento", pena un errore di compilazione.

Relazione di compatibilità (o assegnabilità): è una relazione tra tipi e stabilisce quando è possibile assegnare un valore di un certo tipo `T` ad una variabile di tipo `U`. Si dice che `T` è **assegnabile** a `U` se:

- `T` è sottotipo di `U`, oppure
- `T` ed `U` sono tipi base e c'è una conversione implicita da `T` ad `U`.

La relazione di assegnabilità si applica ai seguenti contesti:

- Assegnazione: `a = exp;`
- Chiamata a metodo: `x.f(exp);`
- Ritorno da metodo: `return exp;`

In base alla definizione di sottotipo, un array di qualunque tipo è sottotipo di "array di `Object`". Consideriamo il seguente esempio:

```
1 String[] arr1 = new String[10];
2 Object[] arr2 = arr1;
3 arr2[0] = new Object();
4 String s = arr1[0];
```

L'esempio risulta corretto per il compilatore. Tuttavia, nell'ultima istruzione assegniamo alla variabile `s`, dichiarata `String`, un oggetto di tipo effettivo `Object`, che non è compatibile.

In effetti a runtime viene sollevata un'eccezione (`ArrayStoreException`) al momento della terza istruzione. Questo perché al runtime gli array "ricordano" il tipo con il quale sono stati creati. La JVM utilizza questa informazione per controllare che gli oggetti inseriti nell'array siano sempre di tipo compatibile con quello dichiarato in origine.

Ad ogni modifica ad un array di riferimenti:

```
1 a[i] = exp;
```

la JVM controlla che il tipo effettivo di `exp` sia sottotipo del tipo con cui è stato creato l'array. In caso contrario, la JVM solleva l'eccezione `ArrayStoreException`.

5.1 I cast

Java permette alcune conversioni esplicite di tipo tramite **cast** (anche dette *coercizioni di tipo*). La sintassi è:

```
1 (T) exp
```

Si può utilizzare un **cast** per effettuare esplicitamente una **promozione**, ma in questo caso il **cast** è superfluo.

Altrimenti si può utilizzare un **cast** per effettuare una **promozione al contrario** (ad esempio da `double` a `int`). In questi casi è facile incorrere a perdite di informazione, infatti sono molto sconsigliate. Al loro posto è opportuno utilizzare i metodi appositi della classe `Math` (come `Math.round`).

Sono **consentiti** dal compilatore i seguenti **cast** tra un tipo riferimento (o array) **A** ad un tipo riferimento (o array) **B**:

1. Se **B** è **supertipo** di **A**:

- Si chiama *upcast*.
- È superfluo, perché i valori di **A** sono di per sé assegnabili al tipo **B**.

2. Se **B** è **sottotipo** di **A**:

- Si chiama *downcast*;
- Al runtime, la JVM controlla che l'oggetto da convertire appartenga effettivamente ad una sottoclasse di **B**. In caso contrario viene sollevata l'eccezione `ClassCastException`.
- Si deve cercare di evitare i downcast perché aggirano il type checking svolto dal compilatore. Se proprio si devono usare, esso andrebbe preceduto da un controllo `instanceof`, che assicuri la correttezza della conversione.

È consentito anche se nessuno dei due è sottotipo dell'altro (in questo caso viene chiamato *sidecast*). Vediamolo nel concreto per capire meglio. Consideriamo un **cast** dal tipo **A** al tipo **B**, ovvero:

```
1  A a = ...
2  B b = (B) a;
```

Se nessuna tra **A** e **B** è un'interfaccia, il **cast** dà errore di compilazione, perché è impossibile che abbia successo a runtime. Se una tra **A** e **B** è un'interfaccia, il **cast** è lecito in compilazione ma a runtime può avere successo o fallire.

5.2 I tipi Wrapper

Per ogni tipo base, Java offre una corrispondente classe che ingloba (*wraps*) un valore di quel tipo in un oggetto. Queste classi, dette appunto **wrapper**, servono per trattare i valori base come se fossero oggetti. Le classi wrapper sono:

- Byte, Short, Integer, Long, Float, Double
- Boolean
- Character
- Void

Tutte le classi wrapper sono:

- **Immutabili**: non è possibile modificare il contenuto di un oggetto;
- **Final**: non è possibile creare sottoclassi.

Le classi wrapper hanno un metodo statico `valueOf` che prende come argomento un valore del tipo base corrispondente alla classe e restituisce un oggetto wrapper che lo ingloba.

Esempio:

```
1  Integer n = Integer.valueOf(3);
2  Double x = Double.valueOf(4.1415);
```

A differenza di un costruttore, l'oggetto restituito da `valueOf` non è *necessariamente nuovo*, ovvero `valueOf` cerca di riciclare gli oggetti già creati (*caching*). Questo non è un problema perché gli oggetti wrapper sono immutabili.

Le sei classi wrapper relative ai tipi numerici estendono la **classe astratta** `Number`. Questa classe prevede sei metodi che estraggono il valore contenuto, convertendolo nel tipo base desiderato. Sono:

```
1 public byte = byteValue();
2 public short = shortValue();
3 public int = intValue();
4 public long = longValue();
5 public float = floatValue();
6 public double = doubleValue();
```

Fino alla versione 1.4 di Java, era necessario convertire esplicitamente i valori dei tipi base in oggetti e viceversa. A partire dalla versione 1.5 questo procedimento è stato autorizzato, introducendo l'**autoboxing** e l'**auto-unboxing**. Grazie a queste funzionalità, il compilatore si occupa di inserire le istruzioni di conversione laddove queste siano necessarie.

Ad esempio, l'istruzione:

```
1 Integer n = 7;
2 // viene convertita in:
3 Integer n = Integer.valueOf(7);
4 // allo stesso modo, l'istruzione:
5 Integer i = n + 7;
6 // viene convertita in:
7 Integer i = Integer.valueOf(n.intValue() + 7);
```

L'*autoboxing* può convertire un'espressione di tipo primitivo in un oggetto del tipo wrapper corrispondente, o di un suo supertipo. Non prende in considerazione le conversioni implicite, ad esempio ciascuna delle seguenti istruzioni provoca un errore di compilazione:

```
1 Double x = 1;
2 Double y = 1.0f;
3 Integer n = (byte) 1;
```

Avviene quando:

1. Si assegna un valore primitivo a una variabile di tipo wrapper;
2. Si passa un valore primitivo a un metodo che si aspetta un oggetto wrapper.

L'*auto-unboxing* è un processo che converte un oggetto wrapper nel tipo primitivo corrispondente. Può essere seguito da una promozione. Ad esempio, le seguenti istruzioni sono corrette:

```
1 Integer i = 7; // autoboxing
2 double d = 1; // auto-unboxing seguito da promozione
```

Avviene quando:

1. Si assegna un oggetto wrapper a una variabile di tipo primitivo;
2. Si passa un oggetto wrapper a un metodo che si aspetta un tipo primitivo;

3. Si utilizza un oggetto wrapper in un'operazione che richiede un tipo primitivo (come un'operazione aritmetica).

Esempio:

```
1 Integer wrapper = 20;
2 int primitivo = wrapper; // auto-unboxing
3
4 // equivale a
5 int primitivo = wrapper.intValue();
6
7 // esempio con operazione
8 Integer a = 5;
9 Integer b = 3;
10 int somma = a + b; // auto-unboxing
```

L'operatore == può dare risultati sorprendenti se usato con i tipi wrapper. Infatti, è facile dimenticare che si tratta di un **confronto tra riferimenti**, come per tutti gli oggetti. Ad esempio:

```
1 Integer a = new Integer(7), b = new Integer(7);
2 System.out.println(a == b); // false, sono oggetti diversi
3
4 Integer a = 7, b = 7;
5 System.out.println(a == b); // true, perche' viene chiamato il metodo
6                               // statico valueOf che riutilizza gli
7                               // oggetti corrispondenti a valori
8                               // piccoli
9
10 Integer a = 700, b = 700;
11 System.out.println(a == b); // false, perche' il metodo valueOf
12                               // riutilizza soltanto gli interi compresi
13                               // tra -127 e 127
```

Morale: confrontare i tipi wrapper con equals e non con ==.

6 Java-02

Binding Dinamico: si intende il meccanismo per cui non è il compilatore, ma la JVM ad avere l'ultima parola su quale metodo invocare in corrispondenza di ciascuna chiamata a metodo. In altri termini, si tratta di stabilire il legame di locazione di un nome di un metodo nel contesto di una invocazione.

Ad esempio:

```
1 x.f(exp);
```

- Il binding di `x` è statico in quanto la sua identità viene stabilita a tempo di compilazione;
- Il binding di `x.f` è dinamico in quanto la sua identità viene stabilita a tempo di esecuzione. Il fatto che il binding sia dinamico è diretta conseguenza del polimorfismo e dell'overriding. Ovvero ciascun riferimento può puntare ad oggetti di tipo effettivo diverso (polimorfismo) e ciascuno di questi tipi effettivi può prevedere una versione diversa dello stesso metodo (overriding).

In Java, il binding dei metodi (collegare ciascuna chiamata ad un metodo vero e proprio) avviene in due fasi:

- **Early binding:** il compilatore risolve l'*overloading*, scegliendo la firma più appropriata alla chiamata;
- **Late binding:** la JVM risolve l'*overriding* (scegliendo il metodo vero e proprio). Non è necessario per metodi che non ammettono overriding (quindi quelli privati, statici o final). Per questi metodi si parla di binding statico.

Molti manuali di Java semplificano le regole dell'overloading, suggerendo che venga scelto il metodo che richiede il *minor numero di conversioni*.

Ad esempio, valutiamo l'invocazione `x.f(1, 2)`, supponendo che la classe di `x` offra i seguenti metodi:

```
1 public void f(double x, long y);
2 public void f(int x, double y);
```

Contiamo il numero di conversioni richieste, in due modi diversi:

1. Primo modo: contiamo quanti parametri richiedono una conversione;
2. Secondo modo: contiamo il numero totale di "passi di conversione" richiesti (cioè, quanti archi dobbiamo percorrere nel grafo che rappresenta le conversioni implicite).

	# parametri da convertire	# passi di conversione
<code>public void f(double x, long y)</code>	2	3
<code>public void f(int d, double l)</code>	1	2

In entrambi i modi, sembra prevalere la seconda firma. Invece, questo è un caso di **ambiguità** (errore di compilazione).

L'early binding si divide a sua volta in due fasi:

1. Individuazione delle firme candidate;
2. Scelta della firma più specifica tra quelle candidate.

6.0.1 Individuazione delle firme candidate

Consideriamo una generica invocazione:

$$x.f(a_1, \dots, a_n)$$

Una generica firma $f(T_1, \dots, T_n)$ è *candidata* per la chiamata in questione se:

- Si trova nella classe dichiarata di x o in una sua superclasse;
- È visibile dal punto della chiamata, rispetto alla regole di visibilità di Java;
- È compatibile con la chiamata; ovvero, per ogni indice compreso tra 1 ed n , il tipo (dichiarato) del parametro attuale a_i è assegnabile al tipo T_i .

Se nessuna firma risulta essere candidata, il compilatore segnala un errore, accompagnato da un messaggio "cannot find symbol".

6.0.2 Confrontare le firme candidate

Date due firme con lo stesso nome e numero di argomenti

$$f(T_1, \dots, T_n) \text{ e } f(U_1, \dots, U_n)$$

Si dice che la prima firma **più specifica** della seconda se, per ogni indice compreso tra 1 ed n , il tipo T_i è assegnabile al tipo U_i .

Attenzione: questo confronto tra firme non dipende dal tipo dei parametri attuali passati alla chiamata.

Questa relazione è una relazione d'ordine, proprio come la relazione di assegnabilità. Tale ordine è parziale, in quanto alcune firme non sono confrontabili tra di loro.

6.0.3 Scelta della firma più specifica

L'early binding si conclude individuando, tra le firme candidate, una che sia **più specifica di tutte le altre**.

Per simulare "a mano" questo meccanismo, nei casi complessi può essere conveniente realizzare un diagramma a **grafo** in cui ci sia un nodo per ciascuna firma candidata ed un arco orientato da un nodo "a" a un nodo "b" quando la firma "a" è più specifica di "b".

Se nel grafo c'è un nodo che ha **archi uscenti diretti verso tutte le altre firme**, quella sarà la firma scelta dal compilatore. Se nessuna firma è più specifica di tutte le altre, il compilatore segnala un errore e termina.

È possibile che si trovi più di una firma più specifica di tutte le altre? Perché? No, perché se accadesse il compilatore non saprebbe quale scegliere. Questo è un grande segnale di ambiguità nel codice e genera un errore.

6.0.4 Late Binding

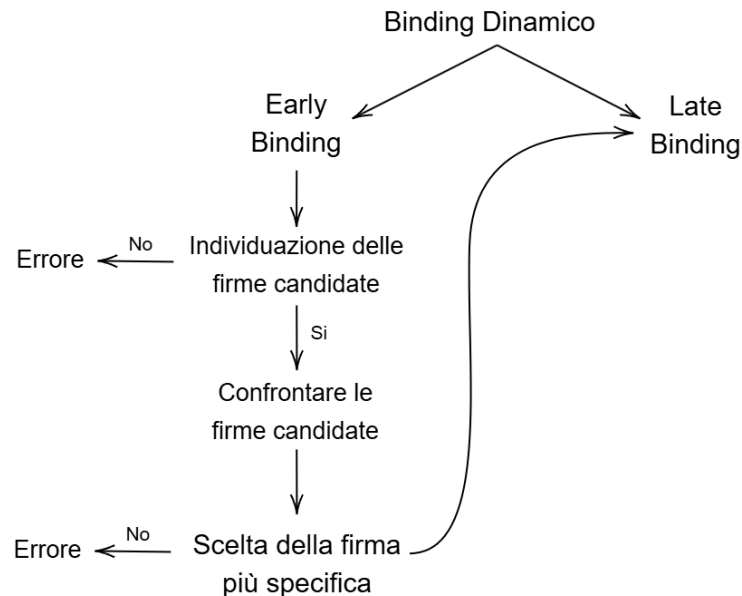
Ora passiamo al late binding. È la fase di risoluzione dell'overriding, a carico della JVM. Questa fase riceve in input la firma scelta dal compilatore durante l'early binding. Consideriamo nuovamente la generica invocazione:

$$x.f(a_1, \dots, a_n)$$

La JVM cerca un metodo da eseguire, con il seguente algoritmo:

- Si parte dalla classe effettiva di x ;
- Si cerca un metodo che abbia la firma **identica** a quella scelta nell'early binding;
- Se non lo si trova, si passa alla superclasse;
- Così via fino ad arrivare a `Object`.

Questo procedimento può fallire, cioè non trovare alcun metodo, solo in casi molto particolari (ad esempio, se una classe `A` dipendeva da una classe `B` e la classe `B` è cambiata da quando è stata compilata `A`).



6.1 Binding dinamico e auto-(un)boxing

Nella risoluzione dell'overloading, l'autoboxing e l'auto-unboxing entrano in gioco *soltanto se necessario* (ovvero soltanto se altrimenti non sarebbero firme candidate).

Quindi, come primo tentativo, il compilatore cerca le firme che sono candidate senza prendere in considerazione l'autoboxing e l'auto-unboxing. Solo se non ci sono firme candidate, il compilatore abilita le conversioni da tipo primitivo a tipo wrapper, e viceversa, e riesamina tutte le firme (secondo tentativo).

Questa scelta è stata fatta per mantenere la compatibilità con il codice scritto prima dell'introduzione dell'auto-(un)boxing (alcune invocazioni che prima non erano consentite, adesso diventano lecite).

Una volta ottenuto un insieme non vuoto di firme candidate, il compilatore passa alla scelta della più specifica, con le regole descritte sopra. Quindi, l'auto-(un)boxing non influenza in nessun modo l'early e il late binding.

Vediamo degli esempi:

```
1 public static int foo(int i, Object o) { return 1; }
2 public static int foo(long i, String o) { return 2; }
```

La chiamata `foo(new Integer(7), "ciao")` provoca un errore di ambiguità perché il compilatore prima ottiene un insieme di firme candidate vuoto; poi, una volta attivato l'auto-(un)boxing, ottiene candidate entrambe le firme `foo`, delle quali nessuna è più specifica dell'altra.

La chiama `foo(new Float(7), "ciao")` provoca un errore di compilazione, in quanto il compilatore non trova firme candidate neanche al secondo tentativo.

La chiama `foo(new Long(7), "ciao")` ottiene output 2, in quanto quella è l'unica firma candidata, una volta attivato l'auto-(un)boxing.

Consideriamo adesso:

```
1 public static int bar(double a, Integer b) { return 3; }
2 public static int bar(Double a, Integer b) { return 4; }
```

La chiamata `bar(1.0, 7)` provoca un errore di ambiguità, perché entrambe le firme saranno candidate (al secondo tentativo).

La chiamata `bar(1,7)` ottiene risultato 3, perché avrà un'unica firma candidata (al secondo tentativo).

7 Java-03

Modificatori: sono parole riservate che danno al compilatore informazioni sulla natura del codice, dei dati, delle classi. Un gruppo di modificatori, detti *di accesso*, specificano in quali contesti lessicali è visibile quell'elemento. Altri modificatori possono essere usati, in combinazione con i precedenti, per qualificare quell'elemento.

7.1 Modificatori di accesso

I modificatori di accesso sono:

- `public`
- `protected`
- `private`

Inoltre, se non è presente alcun modificatore, parleremo di *visibilità di default*.

I seguenti elementi possono essere dotati di modificatore di accesso:

- Classi (anche interfacce ed enumerazioni): regola dove è possibile usare il nome di quella classe (come tipo di una variabile, nell'espressione `new...`)
- Attributi: il modificatore dell'attributo `x` regola quali espressioni della forma `<exp>.x` sono valide.
- Metodi e costruttori: regola quali invocazioni possono avere quel metodo tra le sue firme candidate.

In particolare, le variabili locali non supportano modificatori di accesso, perché la loro visibilità è già limitata al blocco corrente.

I modificatori agiscono solo sui tipi dichiarati (compile time). Significa che quando il compilatore esamina il codice e vede una variabile dichiarata (es. `Animale a`), applica tutte le regole dei modificatori basandosi sul tipo dichiarato (`Animale`), non sul tipo effettivo che quella variabile potrebbe assumere a runtime (`Cane`, `Gatto`, ecc.).

Esempio:

```
1  class Animale {
2      public void mangia() { System.out.println("L'animale mangia."); }
3      private void respira() { System.out.println("L'animale respira."); }
4      protected void dorme() { System.out.println("L'animale dorme."); }
5  }
6
7  class Cane extends Animale {
8      public void abbaia() { System.out.println("Il cane abbaia."); }
9      // Cane ha accesso a mangia() e dorme()
10     // Ma non a respira() di Animale
11 }
12
13 public class Test {
14     public static void main(String[] args) {
15         Animale a = new Cane();
```

```

16      a.mangia(); // OK: mangia() e' public in Animale
17      a.respira(); // ERRORE DI COMPILAZIONE! respira() e' private in Animale
18      // Il compilatore guarda il tipo dichiarato "Animale".
19      // Non gli importa che a sia effettivamente un Cane,
20      // se il metodo e' privato nel tipo dichiarato.
21
22
23      Cane c = new Cane();
24      c.abbaia(); // OK: abbaia() e' public in Cane
25      c.respira(); // ERRORE DI COMPILAZIONE! Anche se c e' un Cane, respira()
26      // e' privato nella sua classe genitore Animale,
27      // e il compilatore lo verifica sul tipo di riferimento.
28  }
29 }

```

Il modificatore `public` è il più permissivo, infatti l'accesso è consentito ovunque.

Il modificatore `private` è il più restrittivo e non si può applicare a classi *top-level*. In questo caso, un attributo/metodo di una classe `A` è accessibile se:

- L'espressione in questione si trova nella classe `A`, oppure
- Il riferimento usato per accedere è di tipo dichiarato `A`.

Con il modificatore `protected` l'accesso è consentito al codice dello stesso pacchetto e a tutte le sottoclassi in pacchetti diversi, ma solo per ereditarietà.

7.2 Altri modificatori

7.2.1 Final

Il modificatore `final` si applica a classi, metodi e variabili (non a costruttori). Il significato varia da contesto a contesto, ma l'essenza è: un elemento `final` non può essere modificato. Inoltre:

- Una classe `final` non può essere estesa, cioè non può avere sottoclassi.
- Una variabile `final` è costante, cioè può solo essere visualizzata.
- Un metodo `final` non può essere sovrascritto (overridden) in una sottoclasse.

Non ha senso attribuire `final` a un costruttore perché esso non è ereditato dalle sottoclassi e quindi non è mai sovrascrivibile.

Attenzione: un riferimento `final` ad un oggetto non può essere modificato (cioè riassegnato a un altro oggetto), ma l'oggetto a cui esso fa riferimento sì.

```

1  final Impiegato luca = new Impiegato("Luca", 1500);
2  luca = new Impiegato("Luca", 1600);      // errore di compilazione
3  luca.setSalario(1600);                  // OK

```

Attenzione: un array `final` non può essere riassegnato, ma il contenuto dell'array può essere modificato.

```

1  final int[] numeri = new int[10];
2  numeri = new int[20];      // errore di compilazione
3  numeri[0] = 77;           // OK

```

7.2.2 Abstract

Il modificatore **abstract** si applica solo a classi e metodi. Un metodo **abstract** non possiede corpo.

```
1  abstract void getValore();
```

Una classe DEVE essere marcata **abstract** se:

- Essa contiene almeno un metodo **abstract**, oppure
- Essa eredita almeno un metodo **abstract** per il quale non fornisce una realizzazione, oppure
- Essa dichiara di implementare una interfaccia, ma non fornisce la realizzazione di tutti i metodi di quella interfaccia.

Qualsiasi classe PUÒ essere marcata **abstract**, anche se non contiene metodi astratti. Il compilatore impedirà di istanziarla.

7.2.3 Static

Il modificatore **static** si applica ad attributi, metodi ed anche a blocchi di codice esterni ai metodi. In generale, una caratteristica **static** appartiene alla classe, non alle singole istanze: essa è unica, indipendentemente dal numero (anche zero) di istanze di quella classe. Questo è il punto più importante: se un attributo o metodo è **static**, non serve creare un oggetto per usarlo. Si può accedere direttamente dalla classe.

L'inizializzazione di un attributo **static** avviene nel momento in cui la classe viene caricata in memoria (anche se non esisterà mai nessuna istanza di quella classe).

```
1  class Ecstatic {
2      static int x = 0;
3      Ecstatic() {x++;}
4  }
```

L'accesso a un attributo **static** di una classe può avvenire (con la dot-notation):

- O partendo da un riferimento ad una istanza di quella classe,
- O partendo dal nome stesso della classe.

```
1  System.out.println(Ecstatic.x);
2  Ecstatic e = new Ecstatic();
3  e.x = 100;
4  Ecstatic.x = 100;
```

I metodi **static** appartengono alla classe e non alle singole istanze. Si possono invocare a partire dal nome della classe:

```
1  class Test {
2      public static void F() {...}
3  }
4  ...
5  Test.f();
```

Non posseggono il riferimento **this**. Hanno binding statico e non possono essere sovrascritti.

Esempio di binding:

```
1  class A {
2      public static void f() {
3          System.out.println("f di A");
4      }
5
6      public void g() {
7          System.out.println("g di A");
8      }
9  }
10
11 class B extends A {
12     // mascheramento, non overriding
13     public static void f() {
14         System.out.println("f di B");
15     }
16
17     @Override
18     public void g() {
19         System.out.println("g di B");
20     }
21 }
22
23 class C {
24     public static void main(...) {
25         A a = new A();
26         B b = new B();
27         A x = b;
28
29         // binding statico
30         A.f(); // stampa "f di A"
31         B.f(); // stampa "f di B"
32         a.f(); // stampa "f di A"
33         b.f(); // stampa "f di B"
34         x.f(); // stampa "f di A"
35
36         // binding dinamico
37         a.g(); // stampa "g di A"
38         b.g(); // stampa "g di B"
39         x.g(); // stampa "g di B"
40     }
41 }
```

Una classe può contenere blocchi di codice marcati **static**. Tali blocchi sono eseguiti una sola volta, nell'ordine in cui compaiono, quando la classe viene caricata in memoria.

```
1  public class EsempioStatic {
2      private static double d = 1.23;
3
4      static {
5          System.out.println("Primo blocco static: d=" + d++);
6      }
7
8      public static void main(String[] args) {
9          System.out.println("main: d=" + d++);
10     }
11 }
```

```
11
12     static {
13         System.out.println("Secondo blocco static: d=" + d++);
14     }
15 }
```

L'uso tipico consiste nell'inizializzare gli attributi statici:

```
1  public class Esempio {
2      private static final int SIZE = 100;
3      private static int[] numeri = new int[SIZE];
4
5      static {
6          for (int i = 0; i < SIZE; i++){
7              numeri[i] = i;
8          }
9      }
10     ...
11 }
```

8 Java-04

8.1 String e StringBuffer

Le stringhe sono oggetti che possono essere creati come segue:

```
1 String s = new String("abcdef");
2 // oppure
3 String s = "abcdef";
```

Sono oggetti **immutabili**. Significa che, una volta assegnato all'oggetto un contenuto, esso è fissato per sempre. Importante specificare che sono immutabili gli oggetti **String**, non i loro riferimenti. Anche i tipi wrapper sono immutabili.

Per motivi di efficienza, poiché nelle applicazioni i letterali **String** occupano molta memoria, la JVM riserva un'area speciale di memoria ad essi: la **String constant pool**.

Quando il compilatore incontra un letterale **String**, esso controlla se è già presente nel pool. Se è presente, allora il letterale viene interpretato come un riferimento all'oggetto **String** esistente. Altrimenti viene creato un nuovo oggetto **String** e aggiunto al pool.

Questo meccanismo di condivisione può funzionare perché gli oggetti **String** sono immutabili. Se lo stesso letterale compare in punti diversi del codice, l'eventuale modifica di un letterale modificherebbe anche l'altro.

Se si deve fare un uso intensivo di manipolazione di stringhe, allora è opportuno usare le classi **StringBuffer** e **StringBuilder**: sono simili a **String** ma sono **mutabili**.

La differenza tra le due è che **StringBuffer** è *thread-safe*. Questo significa che se più parti del programma (chiamati "thread") cercano di modificare la stessa **StringBuffer** contemporaneamente, **StringBuffer** ha dei meccanismi interni che impediscono che si creino problemi (come dati corrotti). **StringBuilder** è più veloce, ma non offre questa protezione.

Per esempio:

```
1 StringBuffer s = new StringBuffer("Walter");
2 s.append(" White");
3 System.out.println(s);    // stampera' Walter White
4 String contenuto = s.toString();
```

Attenzione:

```
1 StringBuffer s = "abc";    // illegale! String e StringBuffer non sono
2                          // auto-convertibili
3
4 StringBuffer s = (StringBuffer) "abc";    // illegale! Neanche con cast
```

Mentre la classe **String** sovrascrive il metodo **equals** in modo da controllare l'uguaglianza del contenuto dei due oggetti (quello corrente e quello ricevuto come parametro), le classi **StringBuffer** e **StringBuilder** non lo sovrascrivono ed usano quello ereditato da **Object**, che funziona come l'operatore **==**.

Le classi **String**, **StringBuffer** e **StringBuilder** sono **final**. Esse non possono essere specializzate.

8.2 Garbage Collection

Adesso il rilascio automatico della memoria non è più accessibile dal programma. Un data object è **eleggibile** per la GC se non è più accessibile dal programma. "Eleggibile per la GC" significa che è diventato un "candidato" per essere rimosso dalla memoria.

Ad esempio:

```
1 public static void main (String[] args) {
2     StringBuffer sb = new StringBuffer("Ciao");
3     System.out.println(sb);
4     sb = null;
5
6     // ora l'oggetto StringBuffer e' eleggibile per la GC
7 }
```

Utilizza un algoritmo **mark-and-sweep**:

- Fase **mark**: a partire dallo stack e dalla regione statica, esplorare tutti gli oggetti accessibili e marcarli come tali. È come la visita di un grafo:
 - I nodi sono i tuoi oggetti.
 - Gli archi sono i riferimenti tra gli oggetti (se un oggetto A contiene un riferimento a un oggetto B, c'è un "arco" da A a B).
 - I nodi di partenza sono i riferimenti sullo stack e nella regione statica.
 - Il GC segue tutti gli archi a partire dai nodi di partenza, visitando e marcando tutti i nodi (oggetti) che incontra.
- Fase **sweep**: In questa fase, il GC scorre tutta l'area della memoria chiamata "heap". Per ogni oggetto nello heap, controlla se è stato "marcato" nella fase precedente. Se un oggetto non è stato marcato, significa che il programma non ha più alcun riferimento ad esso (diretto o indiretto). Il GC a questo punto lo svuota (lo rilascia), rendendo la memoria che occupava disponibile per nuovi oggetti.

Nota: mark-and-sweep è solo uno dei tanti algoritmi possibili di GC; la JVM contiene diversi algoritmi di GC alternativi, selezionabili con opzioni da riga di comando.

Vediamo qualche esempio:

```
1 // Esempio 1
2
3 public static void main (String[] args) {
4     StringBuffer s1 = new StringBuffer("Ciao");
5     StringBuffer s2 = new StringBuffer("Addio");
6     System.out.println(s1);
7     // l'oggetto riferito da s1 non e' ancora eleggibile per GC
8     s1 = s2;
9     // qui e' eleggibile
10    ...
11 }
```



```

1  // Esempio 2
2
3  import java.util.Date;
4  public class TestGC {
5      public static void main(String[] args) {
6          Date d = getDate();
7          System.out.println(d);
8      }
9
10     public static Date getDate(){
11         Date d2 = new Date();
12         String now = d2.toString();
13         System.out.println(now);
14         return d2;
15     }
16 }

```

- In linea 6 il metodo `getDate()` viene chiamato. Lo stack ora include il frame per `getDate()`.
- In linea 11 viene creato un nuovo oggetto `Date` e la variabile `d2` ora referencia questo oggetto (chiamiamolo A). Oggetti eleggibili per GC: Nessuno.
- In linea 12 viene creato un nuovo oggetto `String` (chiamiamolo B) e la variabile `now` lo referencia. Oggetti eleggibili per GC: Nessuno.
- In linea 13 il contenuto dell'oggetto B viene stampato. Oggetti eleggibili per GC: Nessuno.
- In linea 14 la variabile locale `d2` e `now` (nello stack di `getDate()`) stanno per uscire dallo scope. L'oggetto B diventa eleggibile per la GC, mentre oggetto A no in quanto, anche se `d2` sta per scomparire, il suo riferimento viene passato come valore di ritorno.
- In linea 6 il valore di ritorno di `getDate()` viene assegnato alla variabile locale `d` nello stack del `main`. Oggetti eleggibili per GC: Nessuno nuovo.
- In linea 7 il contenuto di oggetto A viene stampato sulla console. Oggetti eleggibili per GC: Nessuno nuovo.
- In linea 8 Il metodo `main` sta per terminare. Oggetti eleggibili per GC: oggetto A.

```

1 // Esempio 3
2
3 public class Isola {
4     Isola i;
5
6     public static void main(String[] args) {
7         Isola i1 = new Isola();    // viene creato l'oggetto A
8         Isola i2 = new Isola();    // viene creato l'oggetto B
9         Isola i3 = new Isola();    // viene creato l'oggetto C
10
11         i1.i = i2;    // A.i = B
12         i2.i = i3;    // B.i = C
13         i3.i = i1;    // C.i = A
14
15         i1 = null;
16         i2 = null;
17         i3 = null;
18
19         // Nonostante A, B e C si riferiscono ancora l'un l'altro
20         // tramite A.i, B.i e C.i, avendo messo i1, i2 e i3 a null,
21         // non sono piu' raggiungibili da nessuna radice. Quindi
22         // tutti gli oggetti sono eleggibili.
23     }
24 }

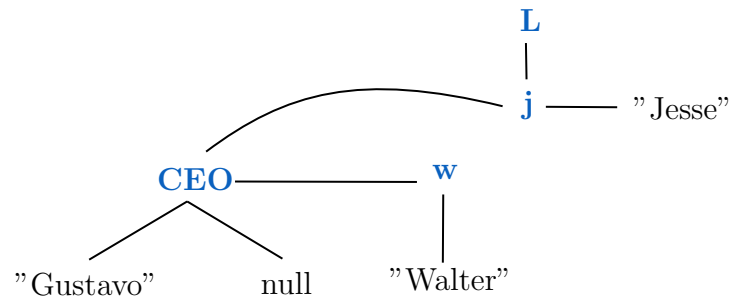
```

```

1 // Esempio 4
2
3 class Employee {
4     private String name;
5     private Employee boss;
6     public final static Employee CEO = new Employee("Gustavo", null);
7     ...
8
9     public static void main(String[] args) {
10         Employee w = new Employee("Walter", CEO);
11         f();
12     }
13
14     public static void f() {
15         Employee j = new Employee("Jesse", CEO),
16             p = new Employee("Pete", j);
17
18         ArrayList<Employee> l = new ArrayList<>();
19         l.add(j);
20         j = null;
21         p = null;
22         //simulare la GC in questo punto
23     }
24
25 }

```

Possiamo rappresentare la fase di mark con questo grafo:



Il processo è questo:

1. Marking da CEO:

- L'oggetto `Employee("Gustavo", null)` referenziato da `CEO` viene marcato.
- Ora, la GC esamina i campi dell'oggetto:
 - `name: "Gustavo"` (è una `String`, anche lei implicitamente marcata perché immutabile e referenziata).
 - `boss: null`. Non c'è un riferimento a un altro oggetto `Employee`.

2. Marking da w:

- L'oggetto `Employee("Walter", CEO)` referenziato da `w` viene marcato.
- Ora, la GC esamina i campi dell'oggetto:
 - `name: "Walter"` (`String`, implicitamente marcata).
 - `boss`: Questo è un riferimento all'oggetto `Employee("Gustavo", null)`. Poiché è già stato marcato, la GC non fa nulla di nuovo qui se non registrare che è già raggiungibile.

3. Marking da l:

- L'oggetto `ArrayList<Employee>` referenziato da `l` viene marcato.
- Ora, la GC esamina il contenuto dell'`ArrayList`:
 - All'interno di `l`, era stato aggiunto l'oggetto a cui `j` puntava inizialmente. Quell'oggetto era `Employee("Jesse", CEO)`. Quindi questo viene marcato.
 - Ora la GC esamina i campi di questo oggetto:
 - * `name: "Jesse"` (`String`, implicitamente marcata).
 - * `boss`: Questo è un riferimento all'oggetto `Employee("Gustavo", null)` già marcato.

Dobbiamo ora passare alla fase *sweep*. Qui:

1. La GC scansiona lo heap.
2. Trova l'oggetto `Employee("Pete", j)` che era stato referenziato in `p`. Dato che questo oggetto non è stato marcato, viene considerato "spazzatura" e la sua memoria viene liberata.

9 Java-05

9.1 Eccezioni: il meccanismo

Le eccezioni denotano "eventi eccezionali" la cui occorrenza altera il flusso normale delle istruzioni. Quando capita un tale evento, si dice che viene "lanciata un'eccezione".

Il linguaggio supporta i seguenti meccanismi relativi alle eccezioni:

- **Lanciare** un'eccezione (istruzione **throw**);
- **Dichiarare** che un metodo lancia un'eccezione (dichiarazione **throws**);
- **Catturare** un'eccezione (blocco **try-catch**).

In Java tutto ciò che non è primitivo è un oggetto. Le eccezioni non fanno "eccezione" a questa regola. Ogni eccezione è un'istanza di una sottoclasse della classe **Throwable**.

Un'eccezione viene lanciata usando la parola riservata **throw**:

```
1 throw <exp>;
```

dove <exp> è un'espressione di tipo dichiarato **Throwable** o suo sottotipo. Esempio:

```
1 throw new IllegalArgumentException();
```

Lanciare un'eccezione interrompe il normale flusso di esecuzione. Se non viene *catturata* localmente, l'eccezione termina il metodo corrente e passa al chiamante, che ha la possibilità di catturarla. Se neanche il metodo chiamante la cattura, l'eccezione continua a risalire lo stack di attivazione, fino a raggiungere il **main**. Se neanche il **main** la cattura, l'eccezione termina il programma e la JVM stampa il contenuto dell'eccezione (*stack trace*).

Quando un'eccezione viene creata, essa registra la situazione corrente dello stack. Esempio:

```
1 public class MyClass {
2     public static void main (String[] args) {
3         (new MyClass()).foo();
4     }
5
6     public void foo() {
7         bar();
8     }
9
10    public void bar() {
11        throw new RuntimeException();
12    }
13 }
```

Output:

```
1 Exception in thread "main" java.lang.RuntimeException
2 at MyClass.bar(MyClass.java:11)
3 at MyClass.foo(MyClass.java:7)
4 at MyClass.main(MyClass.java:3)
```

9.2 Catturare le eccezioni

Per catturare le eccezioni, il codice che potrebbe lanciare eccezioni viene inglobato in un blocco marcato `try`. Il codice che assume la responsabilità di gestire un'eccezione va inglobato in una clausola `catch`.

```
1  try {  
2    // codice "rischioso"  
3  } catch (Eccezione1 e) {  
4    // codice che gestisce una Eccezione1  
5  } catch (Eccezione2 e) {  
6    // codice che gestisce una Eccezione2  
7  }  
8  // codice non rischioso
```

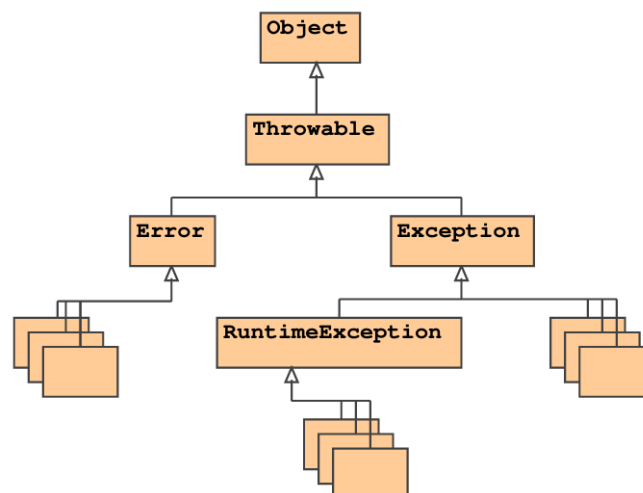
Un blocco (opzionale) marcato `finally` verrà sempre eseguito, anche dopo il lancio e la gestione (eventuale) dell'eccezione.

```
1  try {  
2    // codice "rischioso"  
3  } catch (Eccezione1 e) {  
4    // codice che gestisce una Eccezione1  
5  } catch (Eccezione2 e) {  
6    // codice che gestisce una Eccezione2  
7  } finally {  
8    // codice da eseguire in ogni caso  
9  }  
10 // codice non rischioso
```

Il blocco `finally` viene eseguito perfino dopo una eventuale istruzione `return` presente nei blocchi `try` o `catch`. Il blocco `finally` potrebbe non essere eseguito o potrebbe non completare l'esecuzione solo in conseguenza di un crash totale del sistema oppure tramite una invocazione di `System.exit(int status)`.

Le clausole `catch` e il blocco `finally` sono opzionali, ma ogni `try` deve avere **almeno uno dei due**. Un blocco `try` solitario causa un errore di compilazione.

9.3 Eccezioni: i dettagli



- La classe `Throwable` rappresenta tutti gli oggetti che possono essere lanciati. Essa contiene il metodo `printStackTrace`.
- La classe `Error` e le sue sottoclassi rappresentano situazioni insolite che non sono causate da errori di programmazione o da ciò che normalmente succede durante l'esecuzione del programma. Per esempio: la JVM ha esaurito la memoria oppure qualche risorsa non è più disponibile. Solitamente queste eccezioni non vengono catturate.
- La classe `RuntimeException` rappresenta pure eventi eccezionali, ma dovuti al programma. Il programmatore che si accorge di un baco dovuto ad un suo errore deve correggerlo, non gestirlo. Pertanto, anche queste eccezioni solitamente non vengono catturate.

Una clausola `catch(E e)` cattura ogni oggetto-eccezione il cui tipo effettivo è sottotipo di `E`. Un'eccezione di tipo effettivo `E` viene catturata dal primo blocco `catch` in grado di catturarla.

Eventuali eccezioni lanciate dall'interno dei blocchi `catch` e `finally` non vengono catturate dagli altri blocchi `catch` dello stesso costrutto. Se proprio necessario, i `try-catch` possono essere annidati.

Bisogna resistere alla tentazione di scrivere un'unica clausola `catch-all`:

```
1  try {
2      // codice rischioso
3  } catch (Exception e) {
4      ...
5  }
```

L'ordine delle clausole `catch` è importante.

Le eccezioni si dividono in **verificate** (checked) e **non verificate** (unchecked). Tutte le eccezioni sono verificate, tranne quelle che sono sottoclassi di `Error` e `RuntimeException`. Le eccezioni verificate sono soggette alla regola *handle-or-declare* (vedi dopo).

Come facciamo a sapere che un metodo può lanciare una eccezione che dobbiamo catturare? Così come la dichiarazione del metodo deve specificare il tipo di ritorno, essa può anche specificare le eccezioni che il metodo può lanciare. Si usa la parola chiave `throws`:

```
1  void miaFunzione() throws MiaEccezione1, MiaEccezione2 {
2      ...
3  }
```

Il fatto che un metodo dichiari un'eccezione non significa che la lancerà sempre, ma avverte l'utilizzatore che esso *potrebbe* lanciarla.

Se un metodo lancia un'eccezione verificata, o richiama un altro metodo che può farlo, allora si deve scegliere almeno una di queste opzioni (regola *handle-or-declare*):

1. Catturare e gestire l'eccezione con `try-catch`;
2. Dichiarare l'eccezione nell'intestazione del metodo.

Esempio:

```
1 void f1() {
2     f2();
3 }
4
5 void f2() {
6     throw new IOException();
7 }
```

Il metodo `f2` lancia un'eccezione **checked** ma non la dichiara. Questo è un errore di compilazione. Se esso l'avesse dichiarata, come in:

```
1 void f2() throws IOException {...}
```

il problema l'avrebbe `f1` che dovrebbe ora dichiararla o catturarla.

È possibile usare tipi di eccezioni già presenti nelle Java API, oppure crearne di propri in questo modo:

```
1 class MiaEccezione extends Exception {}
```

oppure estendendo qualsiasi sottoclasse di `Exception`.

9.4 Regole di overriding

Posto che sono sovrascrivibili solo i metodi visibili dalla superclasse, ecco le regole complete per l'overriding dei metodi:

- I due metodi devono avere identica firma;
- Il tipo di ritorno nella sottoclasse può essere uguale o un sottotipo del tipo di ritorno originario;
- Nessuno dei due metodi è **static**.
- Il metodo nella superclasse non può essere marcato **final**;
- Il metodo nella sottoclasse deve avere visibilità non inferiore a quello della superclasse.
- Se il metodo nella sottoclasse dichiara di lanciare un tipo di eccezione checked, tale tipo deve essere un sottotipo di una delle eccezioni dichiarate dal metodo della superclasse.

Nei seguenti casi, una sottoclasse può avere un metodo con la stessa firma di una superclasse, ma non si tratta di overriding:

- Se i due metodi sono entrambi **static**;
- Se il metodo della superclasse non è visibile nella sottoclasse (ad es. perché **private**).

10 Java-06

Java permette di definire una classe (o interfaccia) all'interno di un'altra. Queste classi vengono chiamate **interne** o **annidate** (il termine inglese è *nested*). In particolare, le classi interne non statiche sono chiamate *inner*.

Tale meccanismo arricchisce le possibilità di relazioni tra classi, introducendo in particolare nuove regole di visibilità.

Le classi interne (non statiche) godono delle seguenti proprietà distintive:

- **Privilegi di visibilità** rispetto alla classe contenitrice e alle altre classi in essa contenute. Permettono una stretta collaborazione tra queste classi.
- **Restrizioni di visibilità** rispetto alle classi esterne a quella contenitrice. Permettono di nascondere la classe all'esterno (incapsulamento).
- Un **riferimento implicito** ad un oggetto della classe contenitrice. Ogni oggetto della classe interna "conosce" l'oggetto della classe contenitrice che l'ha creato.

Oltre a campi e metodi, una classe può contenere altre classi o interfacce, dette "interne". Una classe che non sia interna viene chiamata *top-level*. A differenza delle classi top-level, le classi interne possono avere tutte le quattro visibilità ammesse dal linguaggio.

Consideriamo il seguente esempio:

```
1  public class A {  
2      private class B {  
3          ...  
4      }  
5  
6      class C {  
7          ...  
8      }  
9  }
```

La classe B non è visibile al di fuori di A. La classe C, invece, è visibile a tutte le classi che si trovano nello stesso pacchetto di A.

Dall'esterno di A, i nomi completi delle classi B e C sono A.B e A.C, rispettivamente.

La visibilità di una classe interna non ha alcun effetto sul codice che si trova all'interno della classe che la contiene. Ad esempio, la classe B dell'esempio è visibile a tutto il codice contenuto in A, compreso il codice in altre classi interne ad A, come ad esempio C.

In altre parole, tra classi contenute nella stessa classe non vige alcuna restrizione di visibilità.

Ciascun oggetto di una classe interna (non statica) possiede un **riferimento implicito** ad un oggetto della classe contenitrice. Tale riferimento viene inizializzato automaticamente al momento della creazione dell'oggetto e non può essere modificato.

Supponiamo che B sia una classe interna di A. All'interno della classe B, la sintassi per denotare questo riferimento implicito è A.this. Il suo uso è facoltativo, come quello di this, cioè B può accedere ad un campo o metodo "f" della classe A sia con "A.this.f" che con "f".

Esempio:

```
1  public class A {
2      private int x;
3
4      public class B {
5          private int y;
6          public void stampami() {
7              System.out.println(A.this.x);    // uso del riferimento implicito
8              System.out.println(y);
9          }
10     }
11 }
```

Contesto statico: il contesto statico di una classe è quella porzione del codice di quella classe che si trova nell'ambito di validità di un modificatore **static**. Quindi in una delle seguenti posizioni:

- All'interno di un metodo statico;
- All'interno di un blocco di inizializzazione statico;
- Nella definizione di un attributo statico.

La parte restante della classe si chiama **contesto non statico**.

Supponiamo che B sia una classe interna di A. Se viene creato un oggetto di tipo B in un contesto non statico della classe A, il riferimento implicito verrà inizializzato con il valore corrente di **this**. In tutti gli altri casi è necessario utilizzare una nuova forma dell'operatore **new**, ovvero:

<riferimento ad oggetto di tipo A>.new B(...)

Esempio: creazione di un oggetto di classe interna in un *contesto non statico* della classe esterna.

```
1  public class A {
2      private int x;
3
4      public class B {
5          private int y;
6          public void stampami() {
7              System.out.println(A.this.x); // uso del riferimento implicito
8              System.out.println(y);
9          }
10     }
11
12     public B makeB(int val) {
13         B b = new B(); // il nuovo oggetto B e' legato a this
14         b.y = val;    // privilegi di visibilita'
15         return b
16     }
17 }
```

Esempio: creazione di un oggetto di classe interna in un *contesto statico* della classe esterna.

```
1  public class A {
2      private int x;
3
4      public class B {
5          private int y;
```

```

6      public void stampami() {
7          System.out.println(A.this.x); // uso del riferimento implicito
8          System.out.println(y);
9      }
10 }
11
12 public static void main (String[] args) {
13     B b = new B(); // errore di compilazione
14     A a = new A();
15     B b = a.new B(); // OK
16 }
17 }

```

10.1 Classi interne statiche

Le classi interne possono essere statiche o meno. Una classe interna dichiarata nello scope di classe (cioè al di fuori di metodi e inizializzatori) è statica se preceduta dal modificatore **static**. Le classi possono anche trovarsi all'interno di un metodo (parleremo di classi *locali*), ma non ne parleremo.

Le classi interne statiche non possiedono il riferimento implicito alla classe contenitrice.

Il seguente esempio mostra come le classi interne (anche statiche) abbiano appieno accesso ai membri privati della classe contenitrice:

```

1  public class External {
2      private int n = 42;
3
4      public static class A {
5          public void foo(External guest) {
6              // questo e' legale
7              guest.n = 0;
8          }
9      }
10
11     public static class B extends External {
12         public void foo() {
13             // sintassi errata, errore di compilazione
14             n = 0;
15             // sintassi errata, errore di compilazione
16             this.n = 0;
17             // questo e' legale
18             super.n = 0;
19         }
20     }
21 }

```

Se le classi A e B fossero top-level, non potrebbero accedere al campo **n**.

Consideriamo adesso il seguente esempio:

```
1  public class A {  
2      private int x;  
3      private class B {  
4          private int y;  
5      }  
6  
7      static class C {  
8          private double z;  
9      }  
10  
11     public static class D extends A {  
12         private int w;  
13     }  
14  
15     public static void main (String[] args) {  
16         A a = new A();  
17         B b = a.new B();  
18         C c = new C();  
19         A d = new D();  
20     }  
21 }
```

Il suo memory layout è:

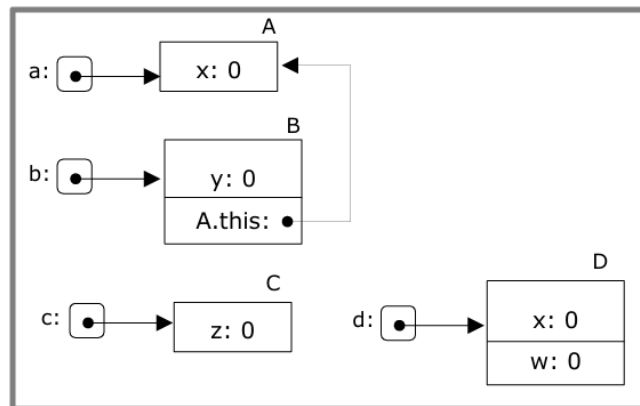


Figura 1: Il memory layout del frammento di programma a sinistra.

In altri linguaggi:

- **C++:**
 - (*Come Java*) Prevede classi interne (nested) con restrizioni di visibilità arbitraria;
 - (*Come Java*) La classe interna ha accesso a tutti i membri della contenitrice;
 - (*Diverso*) La classe interna non ha accesso privilegiato al contenuto di quella interna;
 - (*Diverso*) Non prevede riferimento implicito alla classe esterna;
 - Privilegi di visibilità si ottengono tramite il costrutto **friend**.

- **C#:**
 - Regole analoghe a C++;
 - Nessun costrutto **friend**;
 - Le linee guida MS sconsigliano esplicitamente di creare classi interne pubbliche.

11 Java-08

Vediamo alcuni elementi sintattici per l'inizializzazione di oggetti.

- I **costruttori** (metodi col nome della classe e senza tipo di ritorno);
- Invocazioni ad un costruttore a un altro tipo (**this** e **super**);
- I **blocchi** di inizializzazione (blocchi di codice anonimi nello scope di classe)
- Gli **inizializzatori**.

```
1 public class A {
2     private int n = <exp>;    // inizializzatore
3
4     public A(int a) {        // costruttore
5         n = a;
6     }
7
8     {                        // blocco di inizializzatore
9         n++;
10    }
11
12 }
```

Un costruttore può chiamarne un altro (**chiamata esplicita**) della stessa classe usando la parola chiave **this**, oppure un costruttore della sua superclasse diretta usando la parola chiave **super**. Questi devono essere usati alla **prima riga** del costruttore, pena un errore di compilazione.

Hanno però anche altri significati:

- **this** rappresenta il riferimento all'oggetto corrente;
- **super** si usa per riferirsi a un elemento (metodo o attributo) mascherato appartenente a una superclasse. Ad esempio, per invocare la versione originale di un metodo di cui stiamo facendo l'overriding.

Esempio:

```
1 // l'utente della classe sceglie se fornire un valore o usare quello di
   // default
2
3 class A {
4     private int size;
5
6     public A() {
7         this(1000);    // invoca l'altro costruttore
8     }
9
10    public A (int n) {
11        size = n;
12    }
13 }
```

```

1  public A {
2      public A() {this("Costruttore senza argomenti");}
3      public A(String msg) {System.out.println(msg);}
4  }
5
6  class B extends A {
7      public B() {}
8      public B(int n) {
9          super("Valore: " + n);
10     }
11 }

```

È possibile fare anche **chiamate implicite** ad un altro costruttore. Se un costruttore non inizia con una chiamata ad un altro costruttore (con **this** o **super**), il compilatore inserisce automaticamente una chiamata al costruttore senza argomenti della superclasse. Ovvero inserisce l'istruzione **super()**. In tal caso, se la superclasse non ha un costruttore senza argomenti, si verifica un errore di compilazione.

Il meccanismo tramite il quale i costruttori possono invocarsi a vicenda, prende il nome di **concatenazione di costruttori** (*constructor chaining*).

Sequenza di inizializzazione di un oggetto:

- 1a) Se il costruttore inizia con **super(...)**, passare al costruttore della superclasse (con risoluzione overloading);
- 1b) Se il costruttore inizia con **this(...)**, passare al costruttore indicato (con risoluzione overloading);
- 1c) Se il costruttore non inizia con nessuno dei due, passare al costruttore senza argomenti della superclasse (a meno di non essere **Object**);
- 2) Eseguire i blocchi di inizializzazione e gli inizializzatori degli attributi, nell'ordine in cui compaiono nel codice;
- 3) Eseguire il resto del costruttore;
- 4) Tornare al chiamante.

Nota: un attributo privo di inizializzazione assume il valore di default del suo tipo. Un blocco di inizializzazione o un inizializzatore non può fare un riferimento a un attributo la cui dichiarazione esegue lessicalmente quell'elemento.

Il compilatore controlla che la concatenazione **non sia ciclica**. Infatti, se dei costruttori si chiamano a vicenda, siccome tali chiamate si trovano alla prima riga del rispettivo costruttore, e pertanto avvengono incondizionatamente, ci si trova in presenza di una mutua ricorsione non ben fondata, ovvero infinita.

Ad esempio, tentando di compilare la seguente classe:

```
1  class A {  
2      public A() {  
3          this(3);  
4      }  
5  
6      public A(int i) {  
7          this();  
8      }  
9  }
```

Si ottiene il seguente errore di compilazione:

```
1  A.java:3: recursive constructor invocation  
2      public A(int i) { this(); }
```

Per analizzare la concatenazione dei costruttori di una data gerarchia di classi, ed in particolare controllare che essa non sia ciclica, è possibile realizzare il seguente diagramma:

1. Creare un **grafo** con un nodo per ogni costruttore, compresi quelli impliciti;
2. Aggiungere un arco da un nodo **x** ad un nodo **y** se il costruttore **x** chiama, esplicitamente o implicitamente, il costruttore **y**;
3. Il grafo ottenuto non deve presentare cicli.

Errori di compilazione specifici della costruzione di oggetti sono:

- Concatenazione ciclica di costruttori;
- Fare riferimento, in un iniziatore o in un blocco di inizializzazione, ad un attributo che segue lessicalmente;
- Invocare implicitamente un costruttore della superclasse inesistente.